O'REILLY®

Natural Language Processing with PyTorch

Build Intelligent Language Applications Using Deep Learning

Delip Rao & Brian McMahan

Natural Language Processing with PyTorch

Build Intelligent Language Applications Using Deep Learning

Delip Rao and Brian McMahan

Beijing • Boston • Farnham • Sebastopol • Tokyo

Natural Language Processing with PyTorch

by Delip Rao and Brian McMahan

Printed in the United States of America.

Published by O'Reilly Media, Inc., 1005 Gravenstein Highway North, Sebastopol, CA 95472.

Acquisition Editor: Rachel Roumeliotis	Development Editor: Jeff Bleiel
Production Editor: Nan Barber	Copyeditor: Octal Publishing, LLC
Proofreader: Rachel Head	Indexer: Judy McConville
Interior Designer: David Futato	Cover Designer: Karen Montgomery
Illustrator: Rebecca Demarest	

February 2019: First Edition

Revision History for the First Edition

- 2019-01-16: First Release

See *http://oreilly.com/catalog/errata.csp?isbn=9781491978238* for release details.

used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the authors disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

978-1-491-97823-8

[LSI]

Preface

This book aims to bring newcomers to natural language processing (NLP) and deep learning to a tasting table covering important topics in both areas. Both of these subject areas are growing exponentially. As it introduces both deep learning and NLP with an emphasis on implementation, this book occupies an important middle ground. While writing the book, we had to make difficult, and sometimes uncomfortable, choices on what material to leave out. For a beginner reader, we hope the book will provide a strong foundation in the basics and a glimpse of what is possible. Machine learning, and deep learning in particular, is an experiential discipline, as opposed to an intellectual science. The generous end-to-end code examples in each chapter invite you to partake in that experience.

When we began working on the book, we started with PyTorch 0.2. The examples were revised with each PyTorch update from 0.2 to 0.4. PyTorch 1.0 is due to release around when this book comes out. The code examples in the book are PyTorch 0.4–compliant and should work as they are with the upcoming PyTorch 1.0 release.[1]

A note regarding the style of the book. We have intentionally avoided mathematics in most places, not because deep learning math is particularly difficult (it is not), but because it is a distraction in many situations from the main goal of this book—to empower the beginner learner. Likewise, in many cases, both in code and text, we have favored exposition over succinctness. Advanced readers and experienced programmers will likely see ways to tighten up the code and so on, but our choice was to be as explicit as possible so as to reach the broadest of the audience that we want to reach.

Conventions Used in This Book

The following typographical conventions are used in this book:

Italic

Indicates new terms, URLs, email addresses, filenames, and file extensions.

`Constant width`

Used for program listings, as well as within paragraphs to refer to program elements such as variable or function names, databases, data types, environment variables, statements, and keywords.

`Constant width bold`

Shows commands or other text that should be typed literally by the user.

`Constant width italic`

Shows text that should be replaced with user-supplied values or by values determined by context.

> **TIP**
>
> This element signifies a tip or suggestion.

> **NOTE**
>
> This element signifies a general note.

> **WARNING**
>
> This element indicates a warning or caution.

Using Code Examples

Supplemental material (code examples, exercises, etc.) is available for download at *https://nlproc.info/PyTorchNLPBook/repo/*.

This book is here to help you get your job done. In general, if example code is offered with this book, you may use it in your programs and documentation. You do not need to contact us for permission unless you're reproducing a significant portion of the code. For example, writing a program that uses several chunks of code from this book does not require permission. Selling or distributing a CD-ROM of examples from O'Reilly books does require permission. Answering a question by citing this book and quoting example code does not require permission. Incorporating a significant amount of example code from this book into your product's documentation does require permission.

We appreciate, but do not require, attribution. An attribution usually includes the title, author, publisher, and ISBN. For example: "*Natural Language Processing with PyTorch* by Delip Rao and Brian McMahan (O'Reilly). Copyright 2019, Delip Rao and Brian McMahan, 978-1-491-97823-8."

If you feel your use of code examples falls outside fair use or the permission given above, feel free to contact us at *permissions@oreilly.com*.

O'Reilly Safari

Safari (formerly Safari Books Online) is a membership-based training and reference platform for enterprise, government, educators, and individuals.

Members have access to thousands of books, training videos, Learning Paths, interactive tutorials, and curated playlists from over 250 publishers, including O'Reilly Media, Harvard Business Review, Prentice Hall Professional, Addison-Wesley Professional, Microsoft Press, Sams, Que, Peachpit Press, Adobe, Focal Press, Cisco Press, John Wiley & Sons, Syngress, Morgan Kaufmann, IBM Redbooks, Packt, Adobe Press, FT Press, Apress, Manning, New Riders, McGraw-Hill, Jones & Bartlett, and Course Technology, among others.

For more information, please visit *http://oreilly.com/safari*.

How to Contact Us

Please address comments and questions concerning this book to the publisher:

O’Reilly Media, Inc.

1005 Gravenstein Highway North

Sebastopol, CA 95472

800-998-9938 (in the United States or Canada)

707-829-0515 (international or local)

707-829-0104 (fax)

We have a web page for this book, where we list errata, examples, and any additional information. You can access this page at *http://bit.ly/nlprocbk*.

To comment or ask technical questions about this book, send email to *bookquestions@oreilly.com*.

For more information about our books, courses, conferences, and news, see our website at *http://www.oreilly.com*.

Find us on Facebook: *http://facebook.com/oreilly*

Follow us on Twitter: *http://twitter.com/oreillymedia*

Watch us on YouTube: *http://www.youtube.com/oreillymedia*

Acknowledments

This book has gone through an evolution of sorts, with each version of the book looking unlike the version before. Different folks (and even different DL frameworks) were involved in each version.

The authors want to thank Goku Mohandas for his initial involvement in the book. Goku brought a lot of energy to the project before he had to leave for work reasons. Goku’s enthusiasm for PyTorch and his positivity are

unmatched, and the authors missed his presence. We expect great things coming from him!

The book would not be in top technical form if it not for the kind yet high-quality feedback from our technical reviewers, Liling Tan and Debasish Gosh. Liling contributed his expertise in developing products with state-of-the-art NLP, while Debasish gave highly valuable feedback from the perspective of the developer audience. We are also grateful for the encouragement from Alfredo Canziani, Soumith Chintala, and the many other amazing folks on the PyTorch Developer Forums. We also benefited from the daily rich NLP conversations among the Twitter #nlproc crowd. Many of this book's insights are as much attributable to that community as to our personal practice.

We would be remiss in our duties if we did not express gratitude to Jeff Bleiel for his excellent support as our editor. Without his direction, this book would not have made the light of the day. Bob Russell's copy edits and Nan Barber's production support turned this manuscript from a rough draft into a printable book. We also want to thank Shannon Cutt for her support in the book's early stages.

Much of the material in the book evolved from the 2-day NLP training the authors offered at O'Reilly's AI and Strata conferences. We want to thank Ben Lorica, Jason Perdue, and Sophia DeMartini for working with us on the trainings.

Delip is grateful to have Brian McMahan as a coauthor. Brian went out of his way to support the development of the book. It was a trip to share the joy and pains of development with Brian! Delip also wishes to thank Ben Lorica at O'Reilly for originally insisting he write a book on NLP.

Brian wishes to thank Sara Manuel for her endless support and Delip Rao for being the engine that drove this book to completion. Without Delip's unending persistence and grit, this book would not have been possible.

---

1 See *https://pytorch.org/2018/05/02/road-to-1.0.html*

Chapter 1. Introduction

Household names like Echo (Alexa), Siri, and Google Translate have at least one thing in common. They are all products derived from the application of *natural language processing* (NLP), one of the two main subject matters of this book. NLP refers to a set of techniques involving the application of statistical methods, with or without insights from linguistics, to understand text for the sake of solving real-world tasks. This "understanding" of text is mainly derived by transforming texts to useable computational *representations*, which are discrete or continuous combinatorial structures such as vectors or tensors, graphs, and trees.

The learning of representations suitable for a task from data (text in this case) is the subject of *machine learning*. The application of machine learning to textual data has more than three decades of history, but in the last 10 years[1] a set of machine learning techniques known as *deep learning* have continued to evolve and begun to prove highly effective for various artificial intelligence (AI) tasks in NLP, speech, and computer vision. Deep learning is another main subject that we cover; thus, this book is a study of NLP and deep learning.

> **NOTE**
>
> References are listed at the end of each chapter in this book.

Put simply, deep learning enables one to efficiently learn representations from data using an abstraction called the *computational graph* and numerical optimization techniques. Such is the success of deep learning and computational graphs that major tech companies such as Google, Facebook, and Amazon have published implementations of computational graph frameworks and libraries built on them to capture the mindshare of researchers and engineers. In this book, we consider *PyTorch*, an increasingly popular Python-based computational graph framework to

implement deep learning algorithms. In this chapter, we explain what computational graphs are and our choice of using PyTorch as the framework.

The field of machine learning and deep learning is vast. In this chapter, and for most of this book, we mostly consider what's called *supervised learning*; that is, learning with labeled training examples. We explain the supervised learning paradigm that will become the foundation for the book. If you are not familiar with many of these terms so far, you're in the right place. This chapter, along with future chapters, not only clarifies but also dives deeper into them. If you are already familiar with some of the terminology and concepts mentioned here, we still encourage you to follow along, for two reasons: to establish a shared vocabulary for rest of the book, and to fill any gaps needed to understand the future chapters.

The goals for this chapter are to:

- Develop a clear understanding of the supervised learning paradigm, understand terminology, and develop a conceptual framework to approach learning tasks for future chapters.
- Learn how to encode inputs for the learning tasks.
- Understand what computational graphs are.
- Master the basics of PyTorch.

Let's get started!

The Supervised Learning Paradigm

Supervision in machine learning, or *supervised learning*, refers to cases where the ground truth for the *targets* (what's being predicted) is available for the *observations*. For example, in document classification, the target is a categorical label,[2] and the observation is a document. In machine translation, the observation is a sentence in one language and the target is a sentence in another language. With this understanding of the input data, we illustrate the supervised learning paradigm in Figure 1-1.

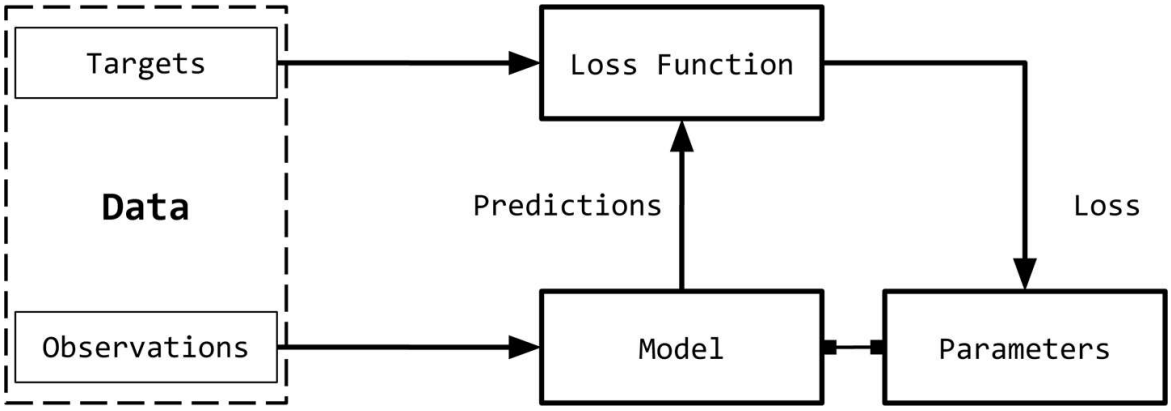


Figure 1-1. The supervised learning paradigm, a conceptual framework for learning from labeled input data.

We can break down the supervised learning paradigm, as illustrated in Figure 1-1, to six main concepts:

Observations

> Observations are items about which we want to predict something. We denote observations using *x*. We sometimes refer to the observations as *inputs*.

Targets

> Targets are labels corresponding to an observation. These are usually the things being predicted. Following standard notations in machine learning/deep learning, we use *y* to refer to these. Sometimes, these labels known as the *ground truth*.

Model

> A model is a mathematical expression or a function that takes an observation, *x*, and predicts the value of its target label.

Parameters

> Sometimes also called *weights*, these parameterize the model. It is standard to use the notation w (for weights) or $\hat{w}$.

Predictions

Predictions, also called *estimates*, are the values of the targets guessed by the model, given the observations. We denote these using a “hat” notation. So, the prediction of a target *y* is denoted as $\hat{y}$.

Loss function

A loss function is a function that compares how far off a prediction is from its target for observations in the training data. Given a target and its prediction, the loss function assigns a scalar real value called the *loss*. The lower the value of the loss, the better the model is at predicting the target. We use *L* to denote the loss function.

Although it is not strictly necessary to be mathematically formal to be productive in NLP/deep learning modeling or to write this book, we will formally restate the supervised learning paradigm to equip readers who are new to the area with the standard terminology so that they have some familiarity with the notations and style of writing in the research papers they may encounter on arXiv.

Consider a dataset $D = \{\mathrm{X}_i, y_i\}_{i=1}^{n}$ with *n* examples. Given this dataset, we want to learn a function (a model) *f* parameterized by weights *w*. That is, we make an assumption about the structure of *f*, and given that structure, the learned values of the weights *w* will fully characterize the model. For a given input X, the model predicts $\hat{y}$ as the target:

$\hat{y} = f(\mathrm{X}, w)$

In supervised learning, for training examples, we know the true target *y* for an observation. The loss for this instance will then be $L(y, \hat{y})$. Supervised learning then becomes a process of finding the optimal parameters/weights *w* that will minimize the cumulative loss for all the *n* examples.

TRAINING USING (STOCHASTIC) GRADIENT DESCENT

The goal of supervised learning is to pick values of the parameters that minimize the loss function for a given dataset. In other words, this is equivalent to finding roots in an equation. We know that *gradient descent* is a common technique to find roots of an equation. Recall that in traditional gradient descent, we guess some initial values for the roots (parameters) and update the parameters iteratively until the objective function (loss function) evaluates to a value below an acceptable threshold (aka convergence criterion). For large datasets, implementation of traditional gradient descent over the entire dataset is usually impossible due to memory constraints, and very slow due to the computational expense. Instead, an approximation for gradient descent called *stochastic gradient descent* (SGD) is usually employed. In the stochastic case, a data point or a subset of data points are picked at random, and the gradient is computed for that subset. When a single data point is used, the approach is called pure SGD, and when a subset of (more than one) data points are used, we refer to it as *minibatch SGD*. Often the words "pure" and "minibatch" are dropped when the approach being used is clear based on the context. In practice, pure SGD is rarely used because it results in very slow convergence due to noisy updates. There are different variants of the general SGD algorithm, all aiming for faster convergence. In later chapters, we explore some of these variants along with how the gradients are used in updating the parameters. This process of iteratively updating the parameters is called *backpropagation*. Each step (aka epoch) of backpropagation consists of a *forward pass* and a *backward pass*. The forward pass evaluates the inputs with the current values of the parameters and computes the loss function. The backward pass updates the parameters using the gradient of the loss.

Observe that until now, nothing here is specific to deep learning or neural networks.[3] The directions of the arrows in Figure 1-1 indicate the "flow" of data while training the system. We will have more to say about training and on the concept of "flow" in "Computational Graphs", but first, let's take a

look at how we can represent our inputs and targets in NLP problems numerically so that we can train models and predict outcomes.

Observation and Target Encoding

We will need to represent the observations (text) numerically to use them in conjunction with machine learning algorithms. Figure 1-2 presents a visual depiction.

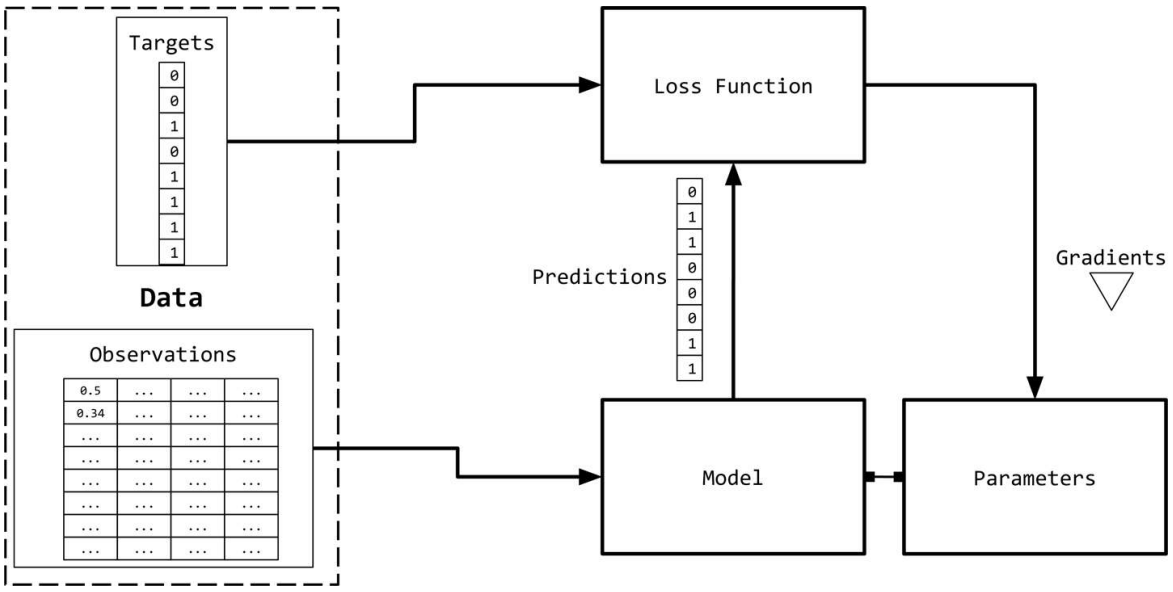


Figure 1-2. Observation and target encoding: The targets and observations from Figure 1-1 are represented numerically as vectors, or tensors. This is collectively known as input "encoding."

A simple way to represent text is as a numerical vector. There are innumerable ways to perform this mapping/representation. In fact, much of this book is dedicated to learning such representations for a task from data. However, we begin with some simple count-based representations that are based on heuristics. Though simple, they are incredibly powerful as they are and can serve as a starting point for richer representation learning. All of these count-based representations start with a vector of fixed dimension.

One-Hot Representation

The *one-hot representation*, as the name suggests, starts with a zero vector, and sets as `1` the corresponding entry in the vector if the word is present in the sentence or document. Consider the following two sentences:

```
Time flies like an arrow.
Fruit flies like a banana.
```

Tokenizing the sentences, ignoring punctuation, and treating everything as lowercase, will yield a vocabulary of size 8: `{time, fruit, flies, like, a, an, arrow, banana}`. So, we can represent each word with an eight-dimensional one-hot vector. In this book, we use $\mathtt{1}_w$ to mean one-hot representation for a token/word *w*.

The collapsed one-hot representation for a phrase, sentence, or a document is simply a logical OR of the one-hot representations of its constituent words. Using the encoding shown in Figure 1-3, the one-hot representation for the phrase “like a banana” will be a 3×8 matrix, where the columns are the eight-dimensional one-hot vectors. It is also common to see a “collapsed” or a binary encoding where the text/phrase is represented by a vector the length of the vocabulary, with `0`s and `1`s to indicate absence or presence of a word. The binary encoding for “like a banana” would then be: `[0, 0, 0, 1, 1, 0, 0, 1]`.

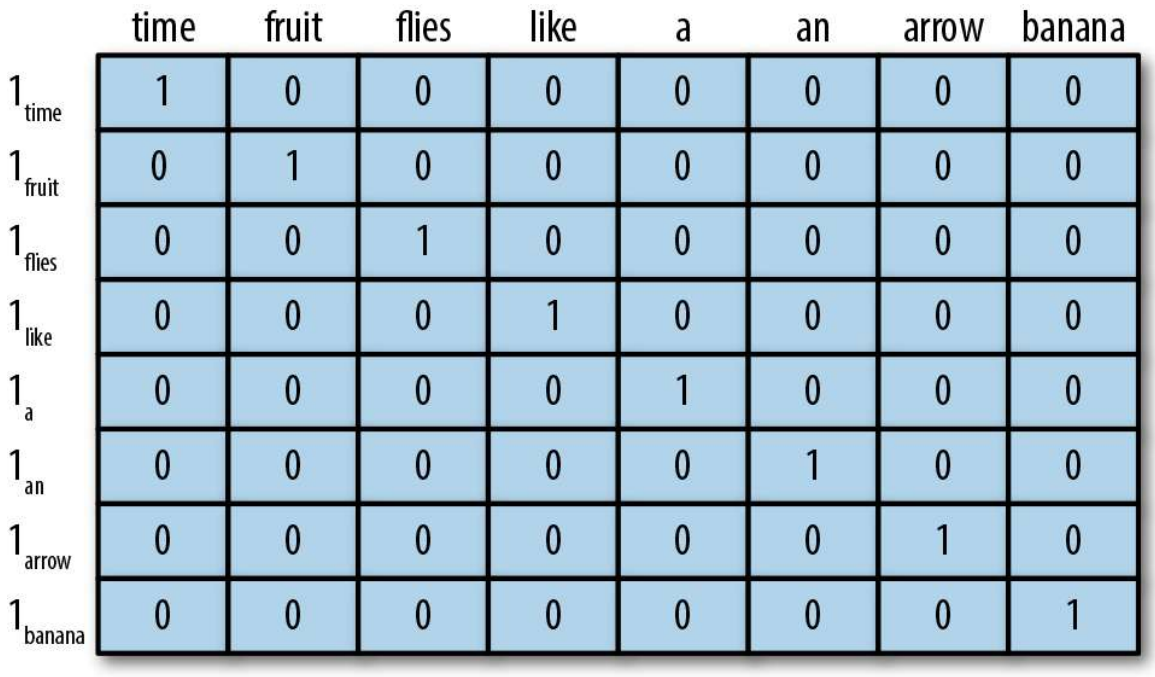


Figure 1-3. One-hot representation for encoding the sentences "Time flies like an arrow" and "Fruit flies like a banana."

> **NOTE**
>
> At this point, if you are cringing that we collapsed the two different meanings (or senses) of "flies," congratulations, astute reader! Language is full of ambiguity, but we can still build useful solutions by making horribly simplifying assumptions. It is possible to learn sense-specific representations, but we are getting ahead of ourselves now.

Although we will rarely use anything other than a one-hot representation for the inputs in this book, we will now introduce the *Term-Frequency* (TF) and *Term-Frequency-Inverse-Document-Frequency* (TF-IDF) representations. This is done because of their popularity in NLP, for historical reasons, and for the sake of completeness. These representations have a long history in information retrieval (IR) and are actively used even today in production NLP systems.

TF Representation

The TF representation of a phrase, sentence, or document is simply the sum of the one-hot representations of its constituent words. To continue with our silly examples, using the aforementioned one-hot encoding, the sentence "Fruit flies like time flies a fruit" has the following TF representation: `[1, 2, 2, 1, 1, 0, 0, 0]`. Notice that each entry is a count of the number of times the corresponding word appears in the sentence (corpus). We denote the TF of a word *w* by *TF(w)*.

Example 1-1. Generating a "collapsed" one-hot or binary representation using scikit-learn

```
from sklearn.feature_extraction.text import CountVectorizer
import seaborn as sns

corpus = ['Time flies flies like an arrow.',
          'Fruit flies like a banana.']
one_hot_vectorizer = CountVectorizer(binary=True)
one_hot = one_hot_vectorizer.fit_transform(corpus).toarray()
sns.heatmap(one_hot, annot=True,
            cbar=False, xticklabels=vocab,
            yticklabels=['Sentence 2'])
```

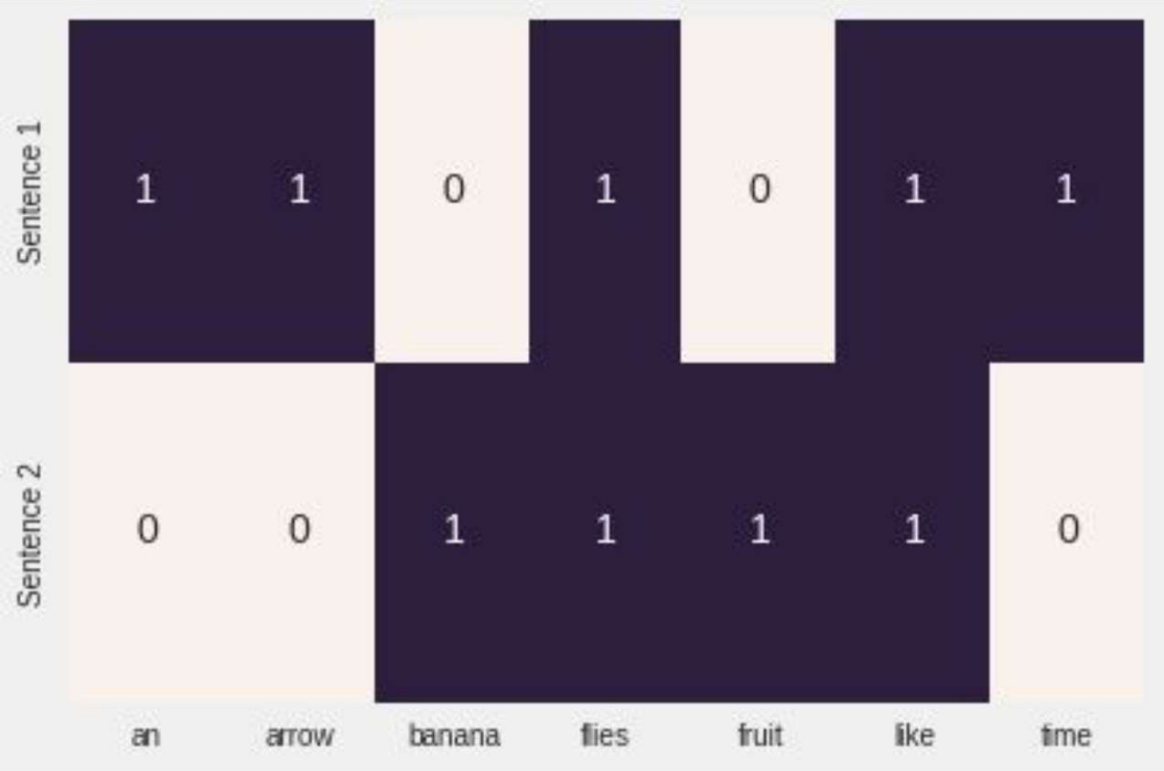


Figure 1-4. The collapsed one-hot representation generated by Example 1-1.

TF-IDF Representation

Consider a collection of patent documents. You would expect most of them to contain words like *claim*, *system*, *method*, *procedure*, and so on, often repeated multiple times. The TF representation weights words proportionally to their frequency. However, common words such as “claim” do not add anything to our understanding of a specific patent. Conversely, if a rare word (such as “tetrafluoroethylene”) occurs less frequently but is quite likely to be indicative of the nature of the patent document, we would want to give it a larger weight in our representation. The Inverse-Document-Frequency (IDF) is a heuristic to do exactly that.

The IDF representation penalizes common tokens and rewards rare tokens in the vector representation. The *IDF(w)* of a token *w* is defined with respect to a corpus as:

$$IDF(w) = \log \frac{N}{n_w}$$

where n_w is the number of documents containing the word *w* and *N* is the total number of documents. The TF-IDF score is simply the product *TF(w) * IDF(w)*. First, notice how if there is a very common word that occurs in all documents (i.e., $n_w = N$), *IDF(w)* is 0 and the TF-IDF score is 0, thereby completely penalizing that term. Second, if a term occurs very rarely, perhaps in only one document, the IDF will be the maximum possible value, log *N*. Example 1-2 shows how to generate a TF-IDF representation of a list of English sentences using `scikit-learn`.

Example 1-2. Generating a TF-IDF representation using scikit-learn

```
from sklearn.feature_extraction.text import TfidfVectorizer
import seaborn as sns

tfidf_vectorizer = TfidfVectorizer()
tfidf = tfidf_vectorizer.fit_transform(corpus).toarray()
sns.heatmap(tfidf, annot=True, cbar=False, xticklabels=vocab,
            yticklabels= ['Sentence 1', 'Sentence 2'])
```

Figure 1-5. The TF-IDF representation generated by Example 1-2.

In deep learning, it is rare to see inputs encoded using heuristic representations like TF-IDF because the goal is to learn a representation. Often, we start with a one-hot encoding using integer indices and a special "embedding lookup" layer to construct inputs to the neural network. In later chapters, we present several examples of doing this.

Target Encoding

As noted in the "The Supervised Learning Paradigm", the exact nature of the target variable can depend on the NLP task being solved. For example, in cases of machine translation, summarization, and question answering, the target is also text and is encoded using approaches such as the previously described one-hot encoding.

Many NLP tasks actually use categorical labels, wherein the model must predict one of a fixed set of labels. A common way to encode this is to use a unique index per label, but this simple representation can become problematic when the number of output labels is simply too large. An example of this is the *language modeling* problem, in which the task is to predict the next word, given the words seen in the past. The label space is the entire vocabulary of a language, which can easily grow to several hundred thousand, including special characters, names, and so on. We revisit this problem in later chapters and see how to address it.

Some NLP problems involve predicting a numerical value from a given text. For example, given an English essay, we might need to assign a numeric grade or a readability score. Given a restaurant review snippet, we might need to predict a numerical star rating up to the first decimal. Given a user's tweets, we might be required to predict the user's age group. Several approaches exist to encode numerical targets, but simply placing the targets into categorical "bins"—for example, "0-18," "19-25," "25-30," and so on—and treating it as an ordinal classification problem is a reasonable approach.[4] The binning can be uniform or nonuniform and data-driven. Although a detailed discussion of this is beyond the scope of this book, we draw your attention to these issues because target encoding affects performance dramatically in such cases, and we encourage you to see Dougherty et al. (1995) and the references therein.

Computational Graphs

Figure 1-1 summarized the supervised learning (training) paradigm as a data flow architecture where the inputs are transformed by the model (a mathematical expression) to obtain predictions, and the loss function (another expression) to provide a feedback signal to adjust the parameters of the model. This data flow can be conveniently implemented using the computational graph data structure.[5] Technically, a computational graph is an abstraction that models mathematical expressions. In the context of deep learning, the implementations of the computational graph (such as Theano, TensorFlow, and PyTorch) do additional bookkeeping to implement automatic differentiation needed to obtain gradients of parameters during training in the supervised learning paradigm. We explore this further in "PyTorch Basics". *Inference* (or prediction) is simply expression evaluation (a forward flow on a computational graph). Let's see how the computational graph models expressions. Consider the expression:

$$y = wx + b$$

This can be written as two subexpressions, $z = wx$ and $y = z + b$. We can then represent the original expression using a directed acyclic graph (DAG) in which the nodes are the mathematical operations, like multiplication and addition. The inputs to the operations are the incoming edges to the nodes and the output of each operation is the outgoing edge. So, for the expression $y = wx + b$, the computational graph is as illustrated in Figure 1-6. In the following section, we see how PyTorch allows us to create computational graphs in a straightforward manner and how it enables us to calculate the gradients without concerning ourselves with any bookkeeping.

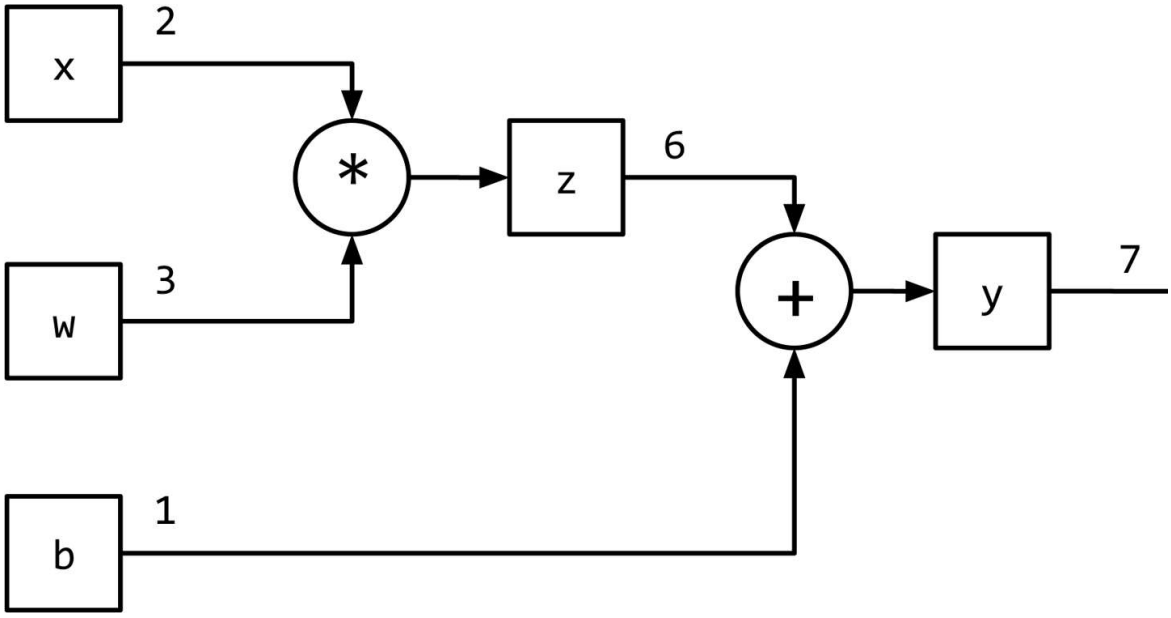


Figure 1-6. Representing y = wx + b using a computational graph.

PyTorch Basics

In this book, we extensively use PyTorch for implementing our deep learning models. PyTorch is an open source, community-driven deep learning framework. Unlike Theano, Caffe, and TensorFlow, PyTorch implements a tape-based automatic differentiation method that allows us to define and execute computational graphs dynamically. This is extremely helpful for debugging and also for constructing sophisticated models with minimal effort.

DYNAMIC VERSUS STATIC COMPUTATIONAL GRAPHS

Static frameworks like Theano, Caffe, and TensorFlow require the computational graph to be first declared, compiled, and then executed.[6] Although this leads to extremely efficient implementations (useful in production and mobile settings), it can become quite cumbersome during research and development. Modern frameworks like Chainer, DyNet, and PyTorch implement dynamic computational graphs to allow for a more flexible, imperative style of development, without needing to compile the models before every execution. Dynamic computational graphs are especially useful in modeling NLP tasks for which each input could potentially result in a different graph structure.

PyTorch is an optimized tensor manipulation library that offers an array of packages for deep learning. At the core of the library is the *tensor*, which is a mathematical object holding some multidimensional data. A tensor of order zero is just a number, or a *scalar*. A tensor of order one (1st-order tensor) is an array of numbers, or a *vector*. Similarly, a 2nd-order tensor is an array of vectors, or a *matrix*. Therefore, a tensor can be generalized as an *n*-dimensional array of scalars, as illustrated in Figure 1-7.

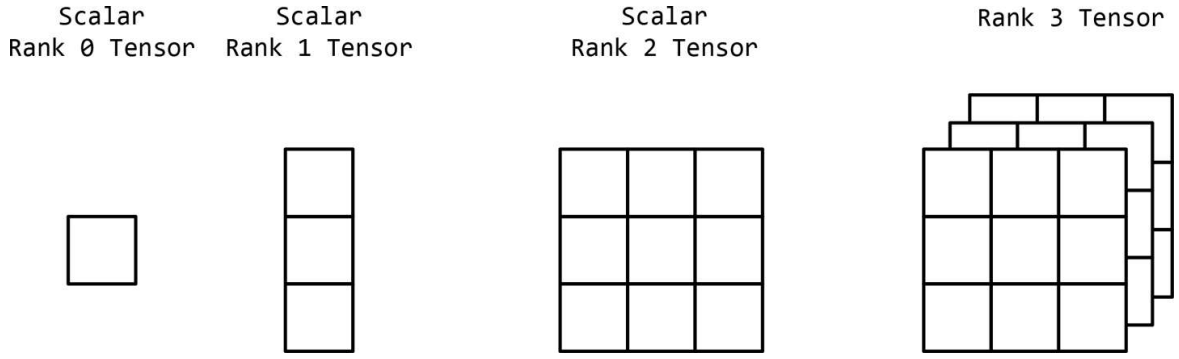


Figure 1-7. Tensors as a generalization of multidimensional arrays.

In this section, we take our first steps with PyTorch to familiarize you with various PyTorch operations. These include:

- Creating tensors
- Operations with tensors

- Indexing, slicing, and joining with tensors
- Computing gradients with tensors
- Using CUDA tensors with GPUs

We recommend that at this point you have a Python 3.5+ notebook ready with PyTorch installed, as described next, and that you follow along with the examples.[7] We also recommend working through the exercises later in the chapter.

Installing PyTorch

The first step is to install PyTorch on your machines by choosing your system preferences at *pytorch.org*. Choose your operating system and then the package manager (we recommend Conda or Pip), followed by the version of Python that you are using (we recommend 3.5+). This will generate the command for you to execute to install PyTorch. As of this writing, the install command for the Conda environment, for example, is as follows:

```
conda install pytorch torchvision -c pytorch
```

> **NOTE**
>
> If you have a CUDA-enabled graphics processor unit (GPU), you should also choose the appropriate version of CUDA. For additional details, follow the installation instructions on *pytorch.org*.

Creating Tensors

First, we define a helper function, `describe(x)`, that will summarize various properties of a tensor *x*, such as the type of the tensor, the dimensions of the tensor, and the contents of the tensor:

```
Input[0]    def describe(x):
                print("Type: {}".format(x.type()))
                print("Shape/size: {}".format(x.shape))
                print("Values: \n{}".format(x))
```

PyTorch allows us to create tensors in many different ways using the `torch` package. One way to create a tensor is to initialize a random one by specifying its dimensions, as shown in Example 1-3.

Example 1-3. Creating a tensor in PyTorch with torch.Tensor

```
Input[0]  import torch
          describe(torch.Tensor(2, 3))
```

```
Output[0] Type: torch.FloatTensor
          Shape/size: torch.Size([2, 3])
          Values:
          tensor([[ 3.2018e-05,  4.5747e-41,  2.5058e+25],
                  [ 3.0813e-41,  4.4842e-44,  0.0000e+00]])
```

We can also create a tensor by randomly initializing it with values from a uniform distribution on the interval `[0, 1)` or the standard normal distribution[8] as illustrated in Example 1-4. Randomly initialized tensors, say from the uniform distribution, are important, as you will see in Chapters 3 and 4.

Example 1-4. Creating a randomly initialized tensor

```
Input[0]  import torch
          describe(torch.rand(2, 3))   # uniform random
          describe(torch.randn(2, 3))  # random normal
```

```
Output[0] Type:  torch.FloatTensor
          Shape/size:  torch.Size([2, 3])
          Values:
           tensor([[ 0.0242,  0.6630,  0.9787],
                   [ 0.1037,   0.3920,  0.6084]])

          Type: torch.FloatTensor
          Shape/size: torch.Size([2, 3])
          Values:
          tensor([[-0.1330, -2.9222, -1.3649],
                  [  2.3648,  1.1561,  1.5042]])
```

We can also create tensors all filled with the same scalar. For creating a tensor of zeros or ones, we have built-in functions, and for filling it with specific values, we can use the `fill_()` method. Any PyTorch method with an underscore (`_`) refers to an in-place operation; that is, it modifies the content in place without creating a new object, as shown in Example 1-5.

Example 1-5. Creating a filled tensor

```
Input[0]  import torch
          describe(torch.zeros(2, 3))
          x = torch.ones(2, 3)
          describe(x)
          x.fill_(5)
          describe(x)
```

```
Output[0] Type: torch.FloatTensor
          Shape/size: torch.Size([2, 3])
          Values:
          tensor([[ 0.,  0.,  0.],
                  [ 0.,  0.,  0.]])

          Type: torch.FloatTensor
          Shape/size: torch.Size([2, 3])
          Values:
          tensor([[ 1.,  1.,  1.],
                  [ 1.,  1.,  1.]])

          Type: torch.FloatTensor
          Shape/size: torch.Size([2, 3])
          Values:
          tensor([[ 5.,  5.,  5.],
                  [ 5.,  5.,  5.]])
```

Example 1-6 demonstrates how we can also create a tensor declaratively by using Python lists.

Example 1-6. Creating and initializing a tensor from lists

```
Input[0]  x = torch.Tensor([[1, 2, 3],
                            [4, 5, 6]])
          describe(x)
```

```
Output[0] Type: torch.FloatTensor
          Shape/size: torch.Size([2, 3])
          Values:
          tensor([[ 1.,  2., 3.],
                  [ 4.,  5., 6.]])
```

The values can either come from a list, as in the preceding example, or from a NumPy array. And, of course, we can always go from a PyTorch tensor to a NumPy array, as well. Notice that the type of the tensor is `DoubleTensor` instead of the default `FloatTensor` (see the next section). This corresponds

with the data type of the NumPy random matrix, a `float64`, as presented in Example 1-7.

Example 1-7. Creating and initializing a tensor from NumPy

```
Input[0]  import torch
          import numpy as np
          npy = np.random.rand(2, 3)
          describe(torch.from_numpy(npy))
```

```
Output[0] Type: torch.DoubleTensor
          Shape/size: torch.Size([2, 3])
          Values:
          tensor([[ 0.8360,  0.8836,  0.0545],
                  [ 0.6928,  0.2333,  0.7984]], dtype=torch.float64)
```

The ability to convert between NumPy arrays and PyTorch tensors becomes important when working with legacy libraries that use NumPy-formatted numerical values.

Tensor Types and Size

Each tensor has an associated type and size. The default tensor type when you use the `torch.Tensor` constructor is `torch.FloatTensor`. However, you can convert a tensor to a different type (`float`, `long`, `double`, etc.) by specifying it at initialization or later using one of the typecasting methods. There are two ways to specify the initialization type: either by directly calling the constructor of a specific tensor type, such as `FloatTensor` or `LongTensor`, or using a special method, `torch.tensor()`, and providing the `dtype`, as shown in Example 1-8.

Example 1-8. Tensor properties

```
Input[0]  x = torch.FloatTensor([[1, 2, 3],
                                 [4, 5, 6]])
          describe(x)
```

```
Output[0] Type: torch.FloatTensor
          Shape/size: torch.Size([2, 3])
          Values:
          tensor([[ 1.,  2.,  3.],
                  [ 4.,  5.,  6.]])
```

```
Input[1]  x = x.long()
          describe(x)
```

```
Output[1] Type: torch.LongTensor
          Shape/size: torch.Size([2, 3])
          Values:
          tensor([[ 1,  2,  3],
                  [ 4,  5,  6]])
```

```
Input[2]  x = torch.tensor([[1, 2, 3],
                            [4, 5, 6]], dtype=torch.int64)
          describe(x)
```

```
Output[2] Type: torch.LongTensor
          Shape/size: torch.Size([2, 3])
          Values:
          tensor([[ 1,  2,  3],
                  [ 4,  5,  6]])
```

```
Input[3]  x = x.float()
          describe(x)
```

```
Output[3] Type: torch.FloatTensor
          Shape/size: torch.Size([2, 3])
          Values:
          tensor([[ 1.,  2.,  3.],
                  [ 4.,  5.,  6.]])
```

We use the `shape` property and `size()` method of a tensor object to access the measurements of its dimensions. The two ways of accessing these measurements are mostly synonymous. Inspecting the shape of the tensor is an indispensable tool in debugging PyTorch code.

Tensor Operations

After you have created your tensors, you can operate on them like you would do with traditional programming language types, like `+`, `-`, `*`, `/`.

Instead of the operators, you can also use functions like `.add()`, as shown in Example 1-9, that correspond to the symbolic operators.

Example 1-9. Tensor operations: addition

```
Input[0]  import torch
          x = torch.randn(2, 3)
          describe(x)

Output[0] Type: torch.FloatTensor
          Shape/size: torch.Size([2, 3])
          Values:
          tensor([[ 0.0461,  0.4024, -1.0115],
                  [ 0.2167, -0.6123,  0.5036]])

Input[1]  describe(torch.add(x, x))

Output[1] Type: torch.FloatTensor
          Shape/size: torch.Size([2, 3])
          Values:
          tensor([[ 0.0923,  0.8048, -2.0231],
                  [ 0.4335, -1.2245,  1.0072]])

Input[2]  describe(x + x)

Output[2] Type: torch.FloatTensor
          Shape/size: torch.Size([2, 3])
          Values:
          tensor([[ 0.0923,  0.8048, -2.0231],
                  [ 0.4335, -1.2245,  1.0072]])
```

There are also operations that you can apply to a specific dimension of a tensor. As you might have already noticed, for a 2D tensor we represent rows as the dimension `0` and columns as dimension `1`, as illustrated in Example 1-10.

Example 1-10. Dimension-based tensor operations

```
Input[0]  import torch
          x = torch.arange(6)
          describe(x)
```

```
Output[0] Type: torch.FloatTensor
          Shape/size: torch.Size([6])
          Values:
          tensor([ 0.,  1.,  2.,  3.,  4.,  5.])
```

```
Input[1]  x = x.view(2, 3)
          describe(x)
```

```
Output[1] Type: torch.FloatTensor
          Shape/size: torch.Size([2, 3])
          Values:
          tensor([[ 0.,  1.,  2.],
                  [ 3.,  4.,  5.]])
```

```
Input[2]  describe(torch.sum(x, dim=0))
```

```
Output[2] Type: torch.FloatTensor
          Shape/size: torch.Size([3])
          Values:
          tensor([ 3.,  5.,  7.])
```

```
Input[3]  describe(torch.sum(x, dim=1))
```

```
Output[3] Type: torch.FloatTensor
          Shape/size: torch.Size([2])
          Values:
          tensor([  3.,  12.])
```

```
Input[4]  describe(torch.transpose(x, 0, 1))
```

```
Output[4] Type: torch.FloatTensor
          Shape/size: torch.Size([3, 2])
          Values:
          tensor([[ 0.,  3.],
                  [ 1.,  4.],
                  [ 2.,  5.]])
```

Often, we need to do more complex operations that involve a combination of indexing, slicing, joining, and mutations. Like NumPy and other numeric libraries, PyTorch has built-in functions to make such tensor manipulations very simple.

Indexing, Slicing, and Joining

If you are a NumPy user, PyTorch's indexing and slicing scheme, shown in Example 1-11, might be very familiar to you.

Example 1-11. Slicing and indexing a tensor

```
Input[0]  import torch
          x = torch.arange(6).view(2, 3)
          describe(x)

Output[0] Type: torch.FloatTensor
          Shape/size: torch.Size([2, 3])
          Values:
          tensor([[ 0.,  1.,  2.],
                  [ 3.,  4.,  5.]])

Input[1]  describe(x[:1, :2])

Output[1] Type: torch.FloatTensor
          Shape/size: torch.Size([1, 2])
          Values:
          tensor([[ 0.,  1.]])

Input[2]  describe(x[0, 1])

Output[2] Type: torch.FloatTensor
          Shape/size: torch.Size([])
          Values:
          1.0
```

Example 1-12 demonstrates that PyTorch also has functions for complex indexing and slicing operations, where you might be interested in accessing noncontiguous locations of a tensor efficiently.

Example 1-12. Complex indexing: noncontiguous indexing of a tensor

```
Input[0]  indices = torch.LongTensor([0, 2])
          describe(torch.index_select(x, dim=1, index=indices))
```

```
Output[0] Type: torch.FloatTensor
          Shape/size: torch.Size([2, 2])
          Values:
          tensor([[ 0.,  2.],
                  [ 3.,  5.]])
```

```
Input[1]  indices = torch.LongTensor([0, 0])
          describe(torch.index_select(x, dim=0, index=indices))
```

```
Output[1] Type: torch.FloatTensor
          Shape/size: torch.Size([2, 3])
          Values:
          tensor([[ 0.,  1.,  2.],
                  [ 0.,  1.,  2.]])
```

```
Input[2]  row_indices = torch.arange(2).long()
          col_indices = torch.LongTensor([0, 1])
          describe(x[row_indices, col_indices])
```

```
Output[2] Type: torch.FloatTensor
          Shape/size: torch.Size([2])
          Values:
          tensor([ 0.,  4.])
```

Notice that the indices are a `LongTensor`; this is a requirement for indexing using PyTorch functions. We can also join tensors using built-in concatenation functions, as shown in Example 1-13, by specifying the tensors and dimension.

Example 1-13. Concatenating tensors

```
Input[0]  import torch
          x = torch.arange(6).view(2,3)
          describe(x)
```

```
Output[0] Type: torch.FloatTensor
          Shape/size: torch.Size([2, 3])
          Values:
          tensor([[ 0.,  1.,  2.],
                  [ 3.,  4.,  5.]])
```

```
Input[1]  describe(torch.cat([x, x], dim=0))
```

```
Output[1] Type: torch.FloatTensor
          Shape/size: torch.Size([4, 3])
          Values:
          tensor([[ 0.,  1.,  2.],
                  [ 3.,  4.,  5.],
                  [ 0.,  1.,  2.],
                  [ 3.,  4.,  5.]])
```

```
Input[2]  describe(torch.cat([x, x], dim=1))
```

```
Output[2] Type: torch.FloatTensor
          Shape/size: torch.Size([2, 6])
          Values:
          tensor([[ 0.,  1.,  2.,  0.,  1.,  2.],
                  [ 3.,  4.,  5.,  3.,  4.,  5.]])
```

```
Input[3]  describe(torch.stack([x, x]))
```

```
Output[3] Type: torch.FloatTensor
          Shape/size: torch.Size([2, 2, 3])
          Values:
          tensor([[[ 0.,  1.,  2.],
                   [ 3.,  4.,  5.]],

                  [[ 0.,  1.,  2.],
                   [ 3.,  4.,  5.]]])
```

PyTorch also implements highly efficient linear algebra operations on tensors, such as multiplication, inverse, and trace, as you can see in Example 1-14.

Example 1-14. Linear algebra on tensors: multiplication

```
Input[0]  import torch
          x1 = torch.arange(6).view(2, 3)
          describe(x1)

Output[0] Type: torch.FloatTensor
          Shape/size: torch.Size([2, 3])
          Values:
          tensor([[ 0.,  1.,  2.],
                  [ 3.,  4.,  5.]])

Input[1]  x2 = torch.ones(3, 2)
          x2[:, 1] += 1
          describe(x2)

Output[1] Type: torch.FloatTensor
          Shape/size: torch.Size([3, 2])
          Values:
          tensor([[ 1.,  2.],
                  [ 1.,  2.],
                  [ 1.,  2.]])

Input[2]  describe(torch.mm(x1, x2))

Output[2] Type: torch.FloatTensor
          Shape/size: torch.Size([2, 2])
          Values:
          tensor([[  3.,   6.],
                  [ 12.,  24.]])
```

So far, we have looked at ways to create and manipulate constant PyTorch tensor objects. Just as a programming language (such as Python) has variables that encapsulate a piece of data and has additional information about that data (like the memory address where it is stored, for example), PyTorch tensors handle the bookkeeping needed for building computational graphs for machine learning simply by enabling a Boolean flag at instantiation time.

Tensors and Computational Graphs

PyTorch `tensor` class encapsulates the data (the tensor itself) and a range of operations, such as algebraic operations, indexing, and reshaping operations. However, as shown in Example 1-15, when the `requires_grad` Boolean flag is set to `True` on a tensor, bookkeeping operations are enabled that can track the gradient at the tensor as well as the gradient function, both

of which are needed to facilitate the gradient-based learning discussed in "The Supervised Learning Paradigm".

Example 1-15. Creating tensors for gradient bookkeeping

```
Input[0]  import torch
          x = torch.ones(2, 2, requires_grad=True)
          describe(x)
          print(x.grad is None)
```

```
Output[0] Type: torch.FloatTensor
          Shape/size: torch.Size([2, 2])
          Values:
          tensor([[ 1.,  1.],
                  [ 1.,  1.]])
          True
```

```
Input[1]  y = (x + 2) * (x + 5) + 3
          describe(y)
          print(x.grad is None)
```

```
Output[1] Type: torch.FloatTensor
          Shape/size: torch.Size([2, 2])
          Values:
          tensor([[ 21.,  21.],
                  [ 21.,  21.]])
          True
```

```
Input[2]  z = y.mean()
          describe(z)
          z.backward()
          print(x.grad is None)
```

```
Output[2] Type: torch.FloatTensor
          Shape/size: torch.Size([])
          Values:
          21.0
          False
```

When you create a tensor with `requires_grad=True`, you are requiring PyTorch to manage bookkeeping information that computes gradients. First, PyTorch will keep track of the values of the forward pass. Then, at the end of the computations, a single scalar is used to compute a backward pass. The backward pass is initiated by using the `backward()` method on a tensor resulting from the evaluation of a loss function. The backward pass

computes a gradient value for a tensor object that participated in the forward pass.

In general, the gradient is a value that represents the slope of a function output with respect to the function input. In the computational graph setting, gradients exist for each parameter in the model and can be thought of as the parameter's contribution to the error signal. In PyTorch, you can access the gradients for the nodes in the computational graph by using the `.grad` member variable. Optimizers use the `.grad` variable to update the values of the parameters.

CUDA Tensors

So far, we have been allocating our tensors on the CPU memory. When doing linear algebra operations, it might make sense to utilize a GPU, if you have one. To use a GPU, you need to first allocate the tensor on the GPU's memory. Access to the GPUs is via a specialized API called CUDA. The CUDA API was created by NVIDIA and is limited to use on only NVIDIA GPUs.[9] PyTorch offers CUDA tensor objects that are indistinguishable in use from the regular CPU-bound tensors except for the way they are allocated internally.

PyTorch makes it very easy to create these CUDA tensors, transfering the tensor from the CPU to the GPU while maintaining its underlying type. The preferred method in PyTorch is to be *device agnostic* and write code that works whether it's on the GPU or the CPU. In Example 1-16, we first check whether a GPU is available by using `torch.cuda.is_available()`, and retrieve the device name with `torch.device()`. Then, all future tensors are instantiated and moved to the target device by using the `.to(device)` method.

Example 1-16. Creating CUDA tensors

```
Input[0]  import torch
          print (torch.cuda.is_available())
```

```
Output[0] True
```

```
Input[1]  # preferred method: device agnostic tensor instantiation
          device = torch.device("cuda" if torch.cuda.is_available() else
          "cpu")
          print (device)
```

```
Output[1] cuda
```

```
Input[2]  x = torch.rand(3, 3).to(device)
          describe(x)
```

```
Output[2] Type: torch.cuda.FloatTensor
          Shape/size: torch.Size([3, 3])
          Values:
          tensor([[ 0.9149,  0.3993,  0.1100],
                  [ 0.2541,  0.4333,  0.4451],
                  [ 0.4966,  0.7865,  0.6604]], device='cuda:0')
```

To operate on CUDA and non-CUDA objects, we need to ensure that they are on the same device. If we don't, the computations will break, as shown in Example 1-17. This situation arises when computing monitoring metrics that aren't part of the computational graph, for instance. When operating on two tensor objects, make sure they're both on the same device.

Example 1-17. Mixing CUDA tensors with CPU-bound tensors

```
Input[0]  y = torch.rand(3, 3)
          x + y
```

```
Output[0] ---------------------------------------------------------------
          -----
          RuntimeError                          Traceback (most recent call
          last)
                1 y = torch.rand(3, 3)
          ----> 2 x + y

          RuntimeError: Expected object of type
          torch.cuda.FloatTensor but found type torch.FloatTensor for
          argument #3 'other'
```

```
Input[1]  cpu_device = torch.device("cpu")
          y = y.to(cpu_device)
          x = x.to(cpu_device)
          x + y
```

```
Output[1] tensor([[ 0.7159,  1.0685,  1.3509],
                  [ 0.3912,  0.2838,  1.3202],
                  [ 0.2967,  0.0420,  0.6559]])
```

Keep in mind that it is expensive to move data back and forth from the GPU. Therefore, the typical procedure involves doing many of the parallelizable computations on the GPU and then transferring just the final result back to the CPU. This will allow you to fully utilize the GPUs. If you have several CUDA-visible devices (i.e., multiple GPUs), the best practice is to use the `CUDA_VISIBLE_DEVICES` environment variable when executing the program, as shown here:

```
CUDA_VISIBLE_DEVICES=0,1,2,3 python main.py
```

We do not cover parallelism and multi-GPU training as a part of this book, but they are essential in scaling experiments and sometimes even to train large models. We recommend that you refer to the PyTorch documentation and discussion forums for additional help and support on this topic.

Exercises

The best way to master a topic is to solve problems. Here are some warm-up exercises. Many of the problems will require going through the official documentation and finding helpful functions.

1. Create a 2D tensor and then add a dimension of size `1` inserted at dimension `0`.
2. Remove the extra dimension you just added to the previous tensor.
3. Create a random tensor of shape `5x3` in the interval `[3, 7)`
4. Create a tensor with values from a normal distribution (mean=`0`, std=`1`).
5. Retrieve the indexes of all the nonzero elements in the tensor `torch.Tensor([1, 1, 1, 0, 1])`.
6. Create a random tensor of size `(3,1)` and then horizontally stack four copies together.
7. Return the batch matrix-matrix product of two three-dimensional matrices `(a=torch.rand(3,4,5), b=torch.rand(3,5,4))`.
8. Return the batch matrix-matrix product of a 3D matrix and a 2D matrix `(a=torch.rand(3,4,5), b=torch.rand(5,4))`.

Solutions

1. ```
   a = torch.rand(3, 3)

   a.unsqueeze(0)
   ```
2. ```
   a.squeeze(0)
   ```
3. ```
   3 + torch.rand(5, 3) * (7 - 3)
   ```
4. ```
   a = torch.rand(3, 3)

   a.normal_()
   ```

5.
```
a = torch.Tensor([1, 1, 1, 0, 1])
torch.nonzero(a)
```
6.
```
a = torch.rand(3, 1)
a.expand(3, 4)
```
7.
```
a = torch.rand(3, 4, 5)
b = torch.rand(3, 5, 4)
torch.bmm(a, b)
```
8.
```
a = torch.rand(3, 4, 5)
b = torch.rand(5, 4)
torch.bmm(a, b.unsqueeze(0).expand(a.size(0),
*b.size()))
```

Summary

In this chapter, we introduced the main topics of this book—natural language processing, or NLP, and deep learning—and developed a detailed understanding of the supervised learning paradigm. You should now be familiar with, or at least aware of, various relevant terms such as observations, targets, models, parameters, predictions, loss functions, representations, learning/training, and inference. You also saw how to encode inputs (observations and targets) for learning tasks using one-hot encoding, and we also examined count-based representations like TF and TF-IDF. We began our journey into PyTorch by first exploring what computational graphs are, then considering static versus dynamic computational graphs and taking a tour of PyTorch's tensor manipulation operations. In Chapter 2, we provide an overview of traditional NLP. These two chapters should lay down the necessary foundation for you if you're new to the book's subject matter and prepare for you for the rest of the chapters.

References

1. PyTorch official API documentation.

2. Dougherty, James, Ron Kohavi, and Mehran Sahami. (1995). "Supervised and Unsupervised Discretization of Continuous Features." *Proceedings of the 12th International Conference on Machine Learning*.

3. Collobert, Ronan, and Jason Weston. (2008). "A Unified Architecture for Natural Language Processing: Deep Neural Networks with Multitask Learning." *Proceedings of the 25th International Conference on Machine Learning.*

---

1 While the history of neural networks and NLP is long and rich, Collobert and Weston (2008) are often credited with pioneering the adoption of modern-style application deep learning to NLP.

2 A categorical variable is one that takes one of a fixed set of values; for example, `{TRUE, FALSE}`, `{VERB, NOUN, ADJECTIVE, ...}`, and so on.

3 Deep learning is distinguished from traditional neural networks as discussed in the literature before 2006 in that it refers to a growing collection of techniques that enabled reliability by adding more layers in the network. We study why this is important in Chapters 3 and 4.

4 An "ordinal" classification is a multiclass classification problem in which there exists a partial order between the labels. In our age example, the category "0–18" comes before "19–25," and so on.

5 Seppo Linnainmaa first introduced the idea of automatic differentiation on computational graphs as a part of his 1970 masters' thesis! Variants of that became the foundation for modern deep learning frameworks like Theano, TensorFlow, and PyTorch.

6 As of v1.7, TensorFlow has an "eager mode" that makes it unnecessary to compile the graph before execution, but static graphs are still the mainstay of TensorFlow.

7 You can find the code for this section under */chapters/chapter_1/PyTorch_Basics.ipynb* in this book's GitHub repo.

8 The standard normal distribution is a normal distribution with mean=`0` and variance=`1`,

9 This means if you have a non-NVIDIA GPU, say AMD or ARM, you're out of luck as of this writing (of course, you can still use PyTorch in CPU mode). However, this might change in the future.

Chapter 2. A Quick Tour of Traditional NLP

Natural language processing (NLP, introduced in the previous chapter) and *computational linguistics* (CL) are two areas of computational study of human language. NLP aims to develop methods for solving practical problems involving language, such as information extraction, automatic speech recognition, machine translation, sentiment analysis, question answering, and summarization. CL, on the other hand, employs computational methods to understand properties of human language. How do we understand language? How do we produce language? How do we learn languages? What relationships do languages have with one another?

In literature, it is common to see a crossover of methods and researchers, from CL to NLP and vice versa. Lessons from CL about language can be used to inform priors in NLP, and statistical and machine learning methods from NLP can be applied to answer questions CL seeks to answer. In fact, some of these questions have ballooned into disciplines of their own, like phonology, morphology, syntax, semantics, and pragmatics.

In this book, we concern ourselves with only NLP, but we borrow ideas routinely from CL as needed. Before we fully vest ourselves into neural network methods for NLP—the focus of the rest of this book—it is worthwhile to review some traditional NLP concepts and methods. That is the goal of this chapter.

If you have some background in NLP, you can skip this chapter, but you might as well stick around for nostalgia and to establish a shared vocabulary for the future.

Corpora, Tokens, and Types

All NLP methods, be they classic or modern, begin with a text dataset, also called a *corpus* (plural: *corpora*). A corpus usually contains raw text (in ASCII or UTF-8) and any metadata associated with the text. The raw text is a sequence of characters (bytes), but most times it is useful to group those

characters into contiguous units called *tokens*. In English, tokens correspond to words and numeric sequences separated by white-space characters or punctuation.

The metadata could be any auxiliary piece of information associated with the text, like identifiers, labels, and timestamps. In machine learning parlance, the text along with its metadata is called an *instance* or *data point*. The corpus (Figure 2-1), a collection of instances, is also known as a *dataset*. Given the heavy machine learning focus of this book, we freely interchange the terms corpus and dataset throughout.

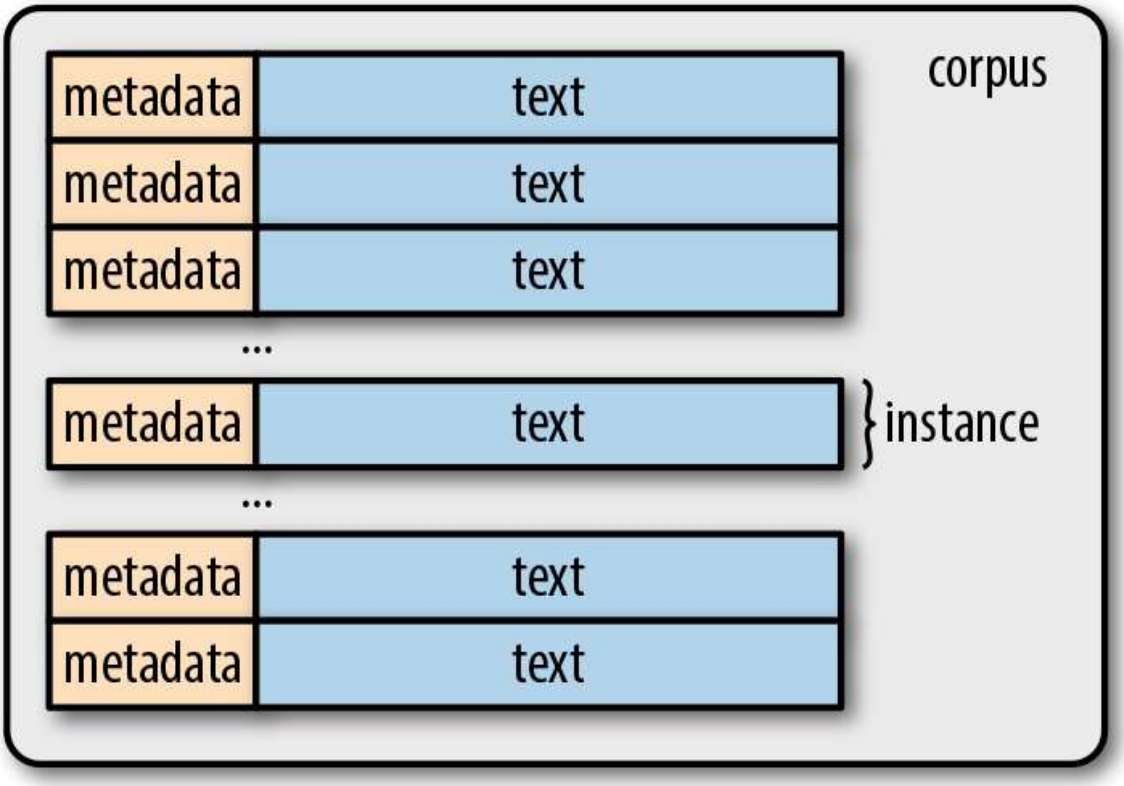


Figure 2-1. The corpus: the starting point of NLP tasks.

The process of breaking a text down into tokens is called *tokenization*. For example, there are six tokens in the Esperanto sentence “Maria frapis la verda sorĉistino.”[1] Tokenization can become more complicated than simply splitting text based on nonalphanumeric characters, as is demonstrated in Figure 2-2. For agglutinative languages like Turkish, splitting on whitespace and punctuation might not be sufficient, and more specialized

techniques might be warranted. As you will see in Chapters 4 and 6, it may be possible to entirely circumvent the issue of tokenization in some neural network models by representing text as a stream of bytes; this becomes very important for agglutinative languages.

Turkish	English
kork(-mak)	(to) fear
korku	fear
korkusuz	fearless
korkusuzlaş (-mak)	(to) become fearless
korkusuzlaşmış	One who has become fearless
korkusuzlaştır(-mak)	(to) make one fearless
korkusuzlaşstırıl(-mak)	(to) be made fearless
korkusuzlaştırılmış	One who has been made fearless
korkusuzlaştırılabil(-mek)	(to) be able to be made fearless
korkusuzlaştırılabilecek	One who will be able to be made fearless
korkusuzlaştırabileceklerimiz	Ones who we can make fearless
korkusuzlaştırabileceklerimizden	From the ones who we can make fearless
korkusuzlaştırabileceklerimizdenmiş	I gather that one is one of those we can make fearless
korkusuzlaştırabileceklerimizdenmişçesine	As if that one is one of those we can make fearless
korkusuzlaştırabileceklerimizdenmişçesineyken	when it seems like that one is one of those we can make fearless

Figure 2-2. Tokenization in languages like Turkish can become complicated quickly.

Finally, consider the following tweet:

Tokenizing tweets involves preserving hashtags and @handles, and segmenting smilies such as `:-)` and URLs as one unit. Should the hashtag #MakeAMovieCold be one token or four? Most research papers don't give much attention to these matters, and in fact, many of the tokenization decisions tend to be arbitrary—but those decisions can significantly affect accuracy in practice more than is acknowledged. Often considered the grunt

work of preprocessing, most open source NLP packages provide reasonable support for tokenization to get you started. Example 2-1 shows examples from NLTK and spaCy, two commonly used packages for text processing.

Example 2-1. Tokenizing text

```
Input[0]  import spacy
          nlp = spacy.load('en')
          text = "Mary, don't slap the green witch"
          print([str(token) for token >in nlp(text.lower())])

Output[0] ['mary', ',', 'do', "n't", 'slap', 'the', 'green', 'witch', '.']

Input[1]  from nltk.tokenize import TweetTokenizer
          tweet=u"Snow White and the Seven Degrees
              #MakeAMovieCold@midnight:-)"
          tokenizer = TweetTokenizer()
          print(tokenizer.tokenize(tweet.lower()))

Output[1] ['snow', 'white', 'and', 'the', 'seven', 'degrees',
           '#makeamoviecold', '@midnight', ':-)']
```

Types are unique tokens present in a corpus. The set of all types in a corpus is its *vocabulary* or *lexicon*. Words can be distinguished as *content words* and *stopwords*. Stopwords such as articles and prepositions serve mostly a grammatical purpose, like filler holding the content words.

FEATURE ENGINEERING

This process of understanding the linguistics of a language and applying it to solving NLP problems is called *feature engineering*. This is something that we keep to a minimum here, for convenience and portability of models across languages. But when building and deploying real-world production systems, feature engineering is indispensable, despite recent claims to the contrary. For an introduction to feature engineering in general, consider reading the book by Zheng and Casari (2016).

Unigrams, Bigrams, Trigrams, …, N-grams

N-grams are fixed-length (*n*) consecutive token sequences occurring in the text. A bigram has two tokens, a unigram one. Generating *n*-grams from a text is straightforward enough, as illustrated in Example 2-2, but packages like spaCy and NLTK provide convenient methods.

Example 2-2. Generating n-grams from text

```
Input[0]  def n_grams(text, n):
              '''
              takes tokens or text, returns a list of n-grams
              '''
              return [text[i:i+n] for i in range(len(text)-n+1)]

          cleaned = ['mary', ',', "n't", 'slap', 'green', 'witch', '.']
          print(n_grams(cleaned, 3))
```

```
Output[0] [['mary', ',', "n't"],
           [',', "n't", 'slap'],
           ["n't", 'slap', 'green'],
           ['slap', 'green', 'witch'],
           ['green', 'witch', '.']]
```

For some situations in which the subword information itself carries useful information, one might want to generate character *n*-grams. For example, the suffix "-ol" in "methanol" indicates it is a kind of alcohol; if your task involved classifying organic compound names, you can see how the subword information captured by *n*-grams can be useful. In such cases, you can reuse the same code, but treat every character *n*-gram as a token.[2]

Lemmas and Stems

Lemmas are root forms of words. Consider the verb *fly*. It can be *inflected* into many different words—*flow*, *flew*, *flies*, *flown*, *flowing*, and so on—and *fly* is the lemma for all of these seemingly different words. Sometimes, it might be useful to reduce the tokens to their lemmas to keep the dimensionality of the vector representation low. This reduction is called *lemmatization*, and you can see it in action in Example 2-3.

Example 2-3. Lemmatization: reducing words to their root forms

```
Input[0]  import spacy
          nlp = spacy.load('en')
          doc = nlp(u"he was running late")
          for token in doc:
              print('{} --> {}'.format(token, token.lemma_))
```

```
Output[0] he --> he
          was --> be
          running --> run
          late --> late
```

spaCy, for example, uses a predefined dictionary, called WordNet, for extracting lemmas, but lemmatization can be framed as a machine learning problem requiring an understanding of the morphology of the language.

Stemming is the poor-man's lemmatization.[3] It involves the use of handcrafted rules to strip endings of words to reduce them to a common form called *stems*. Popular stemmers often implemented in open source packages include the Porter and Snowball stemmers. We leave it to you to find the right spaCy/NLTK APIs to perform stemming.

Categorizing Sentences and Documents

Categorizing or classifying documents is probably one of the earliest applications of NLP. The TF and TF-IDF representations we described in Chapter 1 are immediately useful for classifying and categorizing longer chunks of text such as documents or sentences. Problems such as assigning topic labels, predicting sentiment of reviews, filtering spam emails, language identification, and email triaging can be framed as supervised document classification problems. (Semi-supervised versions, in which only a small labeled dataset is used, are incredibly useful, but that topic is beyond the scope of this book.)

Categorizing Words: POS Tagging

We can extend the concept of labeling from documents to individual words or tokens. A common example of categorizing words is part-of-speech (POS) tagging, as demonstrated in Example 2-4.

Example 2-4. Part-of-speech tagging

```
Input[0]  import spacy
          nlp = spacy.load('en')
          doc = nlp(u"Mary slapped the green witch.")
          for token in doc:
              print('{} - {}'.format(token, token.pos_))
```

```
Output[0] Mary - PROPN
          slapped - VERB
          the - DET
          green - ADJ
          witch - NOUN
          . - PUNCT
```

Categorizing Spans: Chunking and Named Entity Recognition

Often, we need to label a span of text; that is, a contiguous multitoken boundary. For example, consider the sentence, “Mary slapped the green witch.” We might want to identify the noun phrases (NP) and verb phrases (VP) in it, as shown here:

```
[NP Mary] [VP slapped] [the green witch].
```

This is called *chunking* or *shallow parsing*. Shallow parsing aims to derive higher-order units composed of the grammatical atoms, like nouns, verbs, adjectives, and so on. It is possible to write regular expressions over the part-of-speech tags to approximate shallow parsing if you do not have data to train models for shallow parsing. Fortunately, for English and most extensively spoken languages, such data and pretrained models exist. Example 2-5 presents an example of shallow parsing using spaCy.

Example 2-5. Noun Phrase (NP) chunking

```
Input[0]  import spacy
          nlp = spacy.load('en')
          doc  = nlp(u"Mary slapped the green witch.")
          for chunk in doc.noun_chunks:
              print ('{} - {}'.format(chunk, chunk.label_))
```

```
Output[0] Mary - NP
          the green witch - NP
```

Another type of span that's useful is the *named entity*. A named entity is a string mention of a real-world concept like a person, location, organization, drug name, and so on. Here's an example:

Structure of Sentences

Whereas shallow parsing identifies phrasal units, the task of identifying the relationship between them is called *parsing*. You might recall from elementary English class diagramming sentences like in the example shown in Figure 2-3.

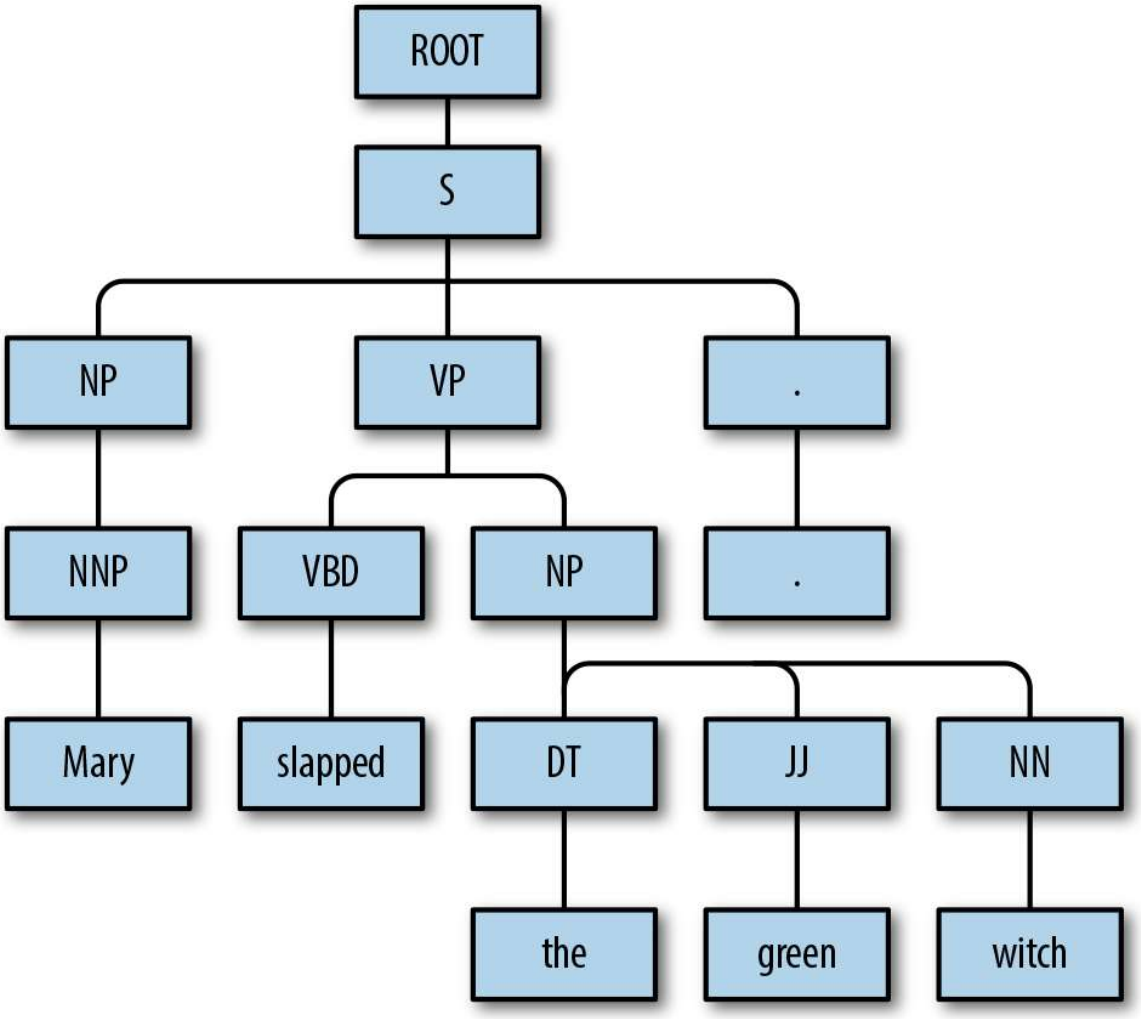


Figure 2-3. A constituent parse of the sentence "Mary slapped the green witch."

Parse trees indicate how different grammatical units in a sentence are related hierarchically. The parse tree in Figure 2-3 shows what's called a *constituent parse*. Another, possibly more useful, way to show relationships is using *dependency parsing*, depicted in Figure 2-4.

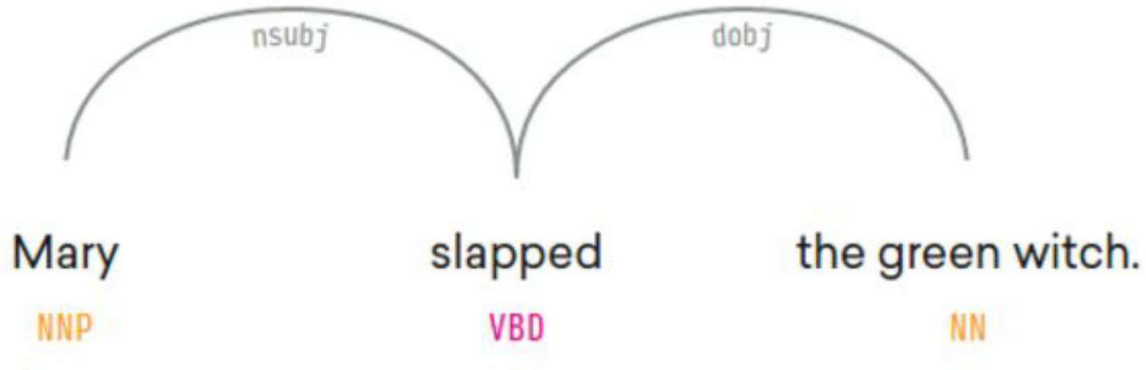


Figure 2-4. A dependency parse of the sentence "Mary slapped the green witch."

To learn more about traditional parsing, see the "References" section at the end of this chapter.

Word Senses and Semantics

Words have meanings, and often more than one. The different meanings of a word are called its *senses*. WordNet, a long-running lexical resource project from Princeton University, aims to catalog the senses of all (well, most) words in the English language, along with other lexical relationships.[4] For example, consider a word like "plane." Figure 2-5 shows the different senses in which this word could be used.

Figure 2-5. Senses for the word "plane" (courtesy of WordNet).

The decades of effort that have been put into projects like WordNet are worth availing yourself of, even in the presence of modern approaches. Later chapters in this book present examples of using existing linguistic resources in the context of neural networks and deep learning methods.

Word senses can also be induced from the context—automatic discovery of word senses from text was actually the first place semi-supervised learning was applied to NLP. Even though we don't cover that in this book, we

encourage you to read Jurafsky and Martin (2014), Chapter 17, and Manning and Schütze (1999), Chapter 7.

Summary

In this chapter, we reviewed some basic terminology and ideas in NLP that should be handy in future chapters. This chapter covered only a smattering of what traditional NLP has to offer. We omitted significant aspects of traditional NLP because we want to allocate the bulk of this book to the use of deep learning for NLP. It is, however, important to know that there is a rich body of NLP research work that doesn't use neural networks, and yet is highly impactful (i.e., used extensively in building production systems). The neural network–based approaches should be considered, in many cases, as a supplement and not a replacement for traditional methods. Experienced practitioners often use the best of both worlds to build state-of-the-art systems. To learn more about the traditional approaches to NLP, we recommend the references listed in the following section.

References

1. Manning, Christopher D., and Hinrich Schütze. (1999). *Foundations of Statistical Natural Language Processing*. MIT press.
2. Bird, Steven, Ewan Klein, and Edward Loper. (2009). *Natural Language Processing with Python: Analyzing Text with the Natural Language Toolkit*. O'Reilly.
3. Smith, Noah A. (2011). *Linguistic Structure prediction.* Morgan and Claypool.
4. Jurafsky, Dan, and James H. Martin. (2014). *Speech and Language Processing*, Vol. 3. Pearson.
5. Russell, Stuart J., and Peter Norvig. (2016). *Artificial Intelligence: A Modern Approach*. Pearson.

6. Zheng, Alice, and Casari, Amanda. (2018). *Feature Engineering for Machine Learning: Principles and Techniques for Data Scientists*. O'Reilly.

---

1 Translation: "Mary slapped the green witch." We use this sentence as a running example in this chapter. We acknowledge the example is rather violent, but our use is a hat-tip to the most famous artificial intelligence textbook of our times (Russell and Norvig, 2016), which also uses this sentence as a running example.

2 In Chapters 4 and 6, we look at deep learning models that implicitly capture this substructure efficiently.

3 To understand the difference between stemming and lemmatization, consider the word "geese." Lemmatization produces "goose," whereas stemming produces "gees."

4 Attempts to create multilingual versions of WordNet exist. See BabelNet as an example.

Chapter 3. Foundational Components of Neural Networks

This chapter sets the stage for later chapters by introducing the basic ideas involved in building neural networks, such as activation functions, loss functions, optimizers, and the supervised training setup. We begin by looking at the perceptron, a one-unit neural network, to tie together the various concepts. The perceptron itself is a building block in more complex neural networks. This is a common pattern that will repeat itself throughout the book—every architecture or network we discuss can be used either standalone or compositionally within other complex networks. This compositionality will become clear as we discuss computational graphs and the rest of this book.

The Perceptron: The Simplest Neural Network

The simplest neural network unit is a *perceptron*. The perceptron was historically and very loosely modeled after the biological neuron. As with a biological neuron, there is input and output, and "signals" flow from the inputs to the outputs, as illustrated in Figure 3-1.

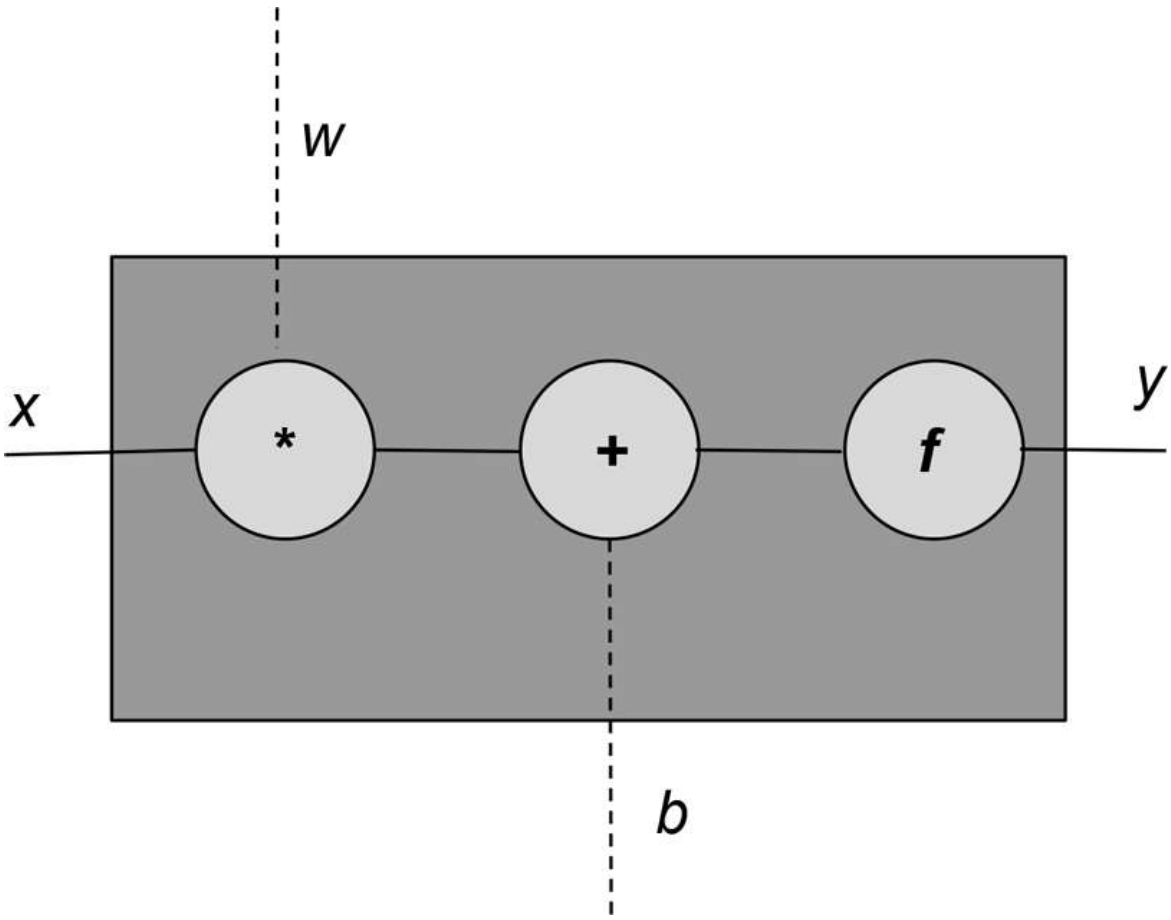


Figure 3-1. The computational graph for a perceptron with an input (x) and an output (y). The weights (w) and bias (b) constitute the parameters of the model.

Each perceptron unit has an input (*x*), an output (*y*), and three "knobs": a set of weights (*w*), a bias (*b*), and an activation function (*f*). The weights and the bias are learned from the data, and the activation function is handpicked depending on the network designer's intuition of the network and its target outputs. Mathematically, we can express this as follows:

$$y = f(\boldsymbol{w}x + \boldsymbol{b})$$

It is usually the case that there is more than one input to the perceptron. We can represent this general case using vectors. That is, $\boldsymbol{x}$, and $\boldsymbol{w}$ are vectors, and the product of $\boldsymbol{w}$ and $\boldsymbol{x}$ is replaced with a dot product:

$$y = f(\boldsymbol{w}\boldsymbol{x} + \boldsymbol{b})$$

The activation function, denoted here by *f*, is typically a nonlinear function. A linear function is one whose graph is a straight line. In this example, ***wx***+***b*** is a linear function. So, essentially, a perceptron is a composition of a linear and a nonlinear function. The linear expression ***wx***+***b*** is also known as an *affine transform*.

Example 3-1 presents a perceptron implementation in PyTorch that takes an arbitrary number of inputs, does the affine transform, applies an activation function, and produces a single output.

Example 3-1. Implementing a perceptron using PyTorch

```
import torch
import torch.nn as nn

class Perceptron(nn.Module):
    """ A perceptron is one linear layer """
    def __init__(self, input_dim):
        """
        Args:
            input_dim (int): size of the input features
        """
        super(Perceptron, self).__init__()
        self.fc1 = nn.Linear(input_dim, 1)

    def forward(self, x_in):
        """The forward pass of the perceptron

        Args:
            x_in (torch.Tensor): an input data tensor
                x_in.shape should be (batch, num_features)
        Returns:
            the resulting tensor. tensor.shape should be (batch,).
        """
        return torch.sigmoid(self.fc1(x_in)).squeeze()
```

PyTorch conveniently offers a `Linear` class in the `torch.nn` module that does the bookkeeping needed for the weights and biases, and does the needed affine transform.[1] In "Diving Deep into Supervised Training", you'll see how to "learn" the values of the weights ***w*** and *b* from data. The activation function used in the preceding example is the *sigmoid function*.

In the following section, we review some common activation functions, including this one.

Activation Functions

Activation functions are nonlinearities introduced in a neural network to capture complex relationships in data. In “Diving Deep into Supervised Training” and “The Multilayer Perceptron” we dive deeper into why nonlinearities are required in the learning, but first, let’s look at a few commonly used activation functions.[2]

Sigmoid

The sigmoid is one of the earliest used activation functions in neural network history. It takes any real value and squashes it into the range between `0` and `1`. Mathematically, the sigmoid function is expressed as follows:

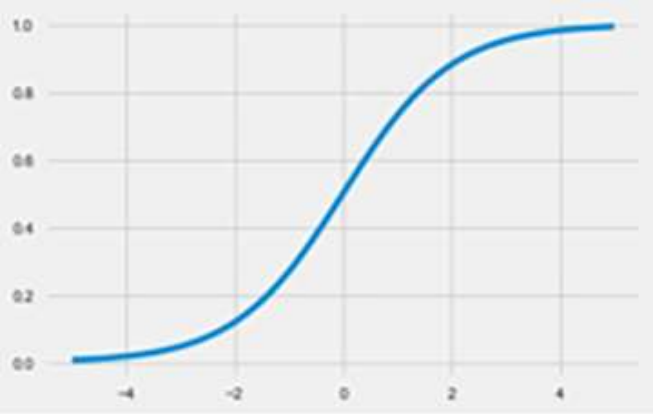
$$f(x) = \frac{1}{1 + e^{-x}}$$

It is easy to see from the expression that the sigmoid is a smooth, differentiable function. `torch` implements the sigmoid as `torch.sigmoid()`, as shown in Example 3-2.

Example 3-2. Sigmoid activation

```
import torch
import matplotlib.pyplot as
plt

x = torch.range(-5., 5., 0.1)
y = torch.sigmoid(x)
plt.plot(x.numpy(),
y.numpy())
plt.show()
```



As you can observe from the plot, the sigmoid function saturates (i.e., produces extreme valued outputs) very quickly and for a majority of the inputs. This can become a problem because it can lead to the gradients becoming either zero or diverging to an overflowing floating-point value. These phenomena are also known as *vanishing gradient problem* and *exploding gradient problem*, respectively. As a consequence, it is rare to see sigmoid units used in neural networks other than at the output, where the squashing property allows one to interpret outputs as probabilities.

Tanh

The tanh activation function is a cosmetically different variant of the sigmoid. This becomes clear when you write down the expression for tanh:

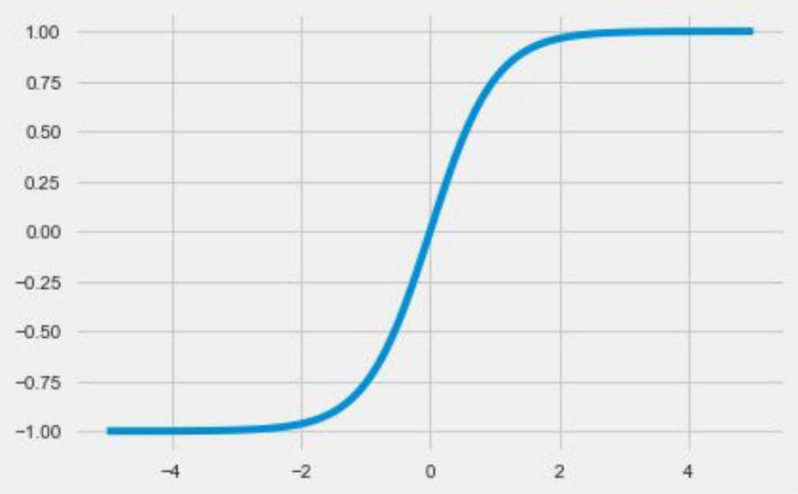
$$f(x) = \tanh x = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$$

With a little bit of wrangling (which we leave for you as an exercise), you can convince yourself that tanh is simply a linear transform of the sigmoid function, as shown in Example 3-3. This is also evident when you write down the PyTorch code for `tanh()` and plot the curve. Notice that tanh, like the sigmoid, is also a "squashing" function, except that it maps the set of real values from (–∞, +∞) to the range `[-1, +1]`.

Example 3-3. Tanh activation

```
import torch
import
matplotlib.pyplot
as plt

x =
torch.range(-5.,
5., 0.1)
y = torch.tanh(x)
plt.plot(x.numpy(),
y.numpy())
plt.show()
```



ReLU

ReLU (pronounced ray-luh) stands for *rectified linear unit*. This is arguably the most important of the activation functions. In fact, one could venture as far as to say that many of the recent innovations in deep learning would've been impossible without the use of ReLU. For something so fundamental, it's also surprisingly new as far as neural network activation functions go. And it's surprisingly simple in form:

$$f(x) = \max(0, x)$$

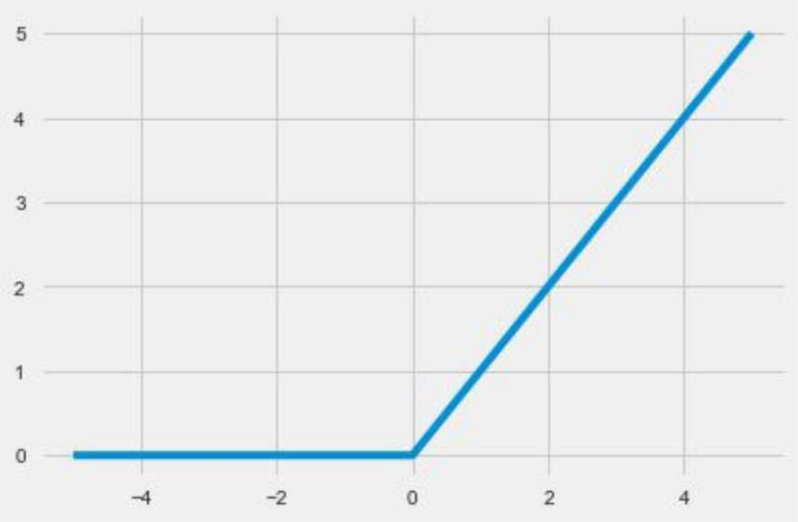
So, all a ReLU unit is doing is clipping the negative values to zero, as demonstrated in Example 3-4.

Example 3-4. ReLU activation

```
import torch
import
matplotlib.pyplot
as plt

relu =
torch.nn.ReLU()
x =
torch.range(-5.,
5., 0.1)
y = relu(x)

plt.plot(x.numpy(),
y.numpy())
plt.show()
```



The clipping effect of ReLU that helps with the vanishing gradient problem can also become an issue, where over time certain outputs in the network can simply become zero and never revive again. This is called the "dying ReLU" problem. To mitigate that effect, variants such as the Leaky ReLU and Parametric ReLU (PReLU) activation functions have proposed, where the leak coefficient *a* is a learned parameter. Example 3-5 shows the result.

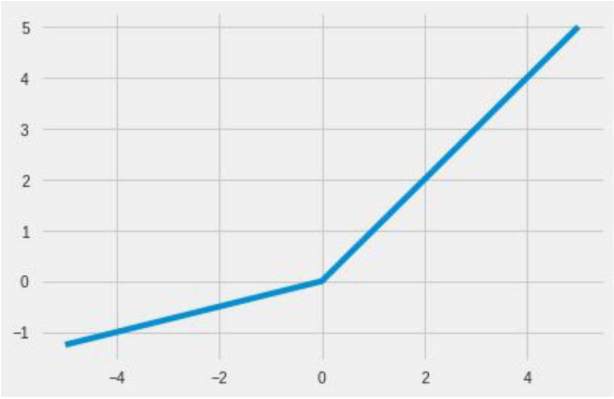
$$f(x) = \max(x, ax)$$

Example 3-5. PReLU activation

```
import torch
import matplotlib.pyplot as plt

prelu =
torch.nn.PReLU(num_parameters=1)
x = torch.range(-5., 5., 0.1)
y = prelu(x)

plt.plot(x.numpy(), y.numpy())
plt.show()
```



Softmax

Another choice for the activation function is the softmax. Like the sigmoid function, the softmax function squashes the output of each unit to be between `0` and `1`, as shown in Example 3-6. However, the softmax operation also divides each output by the sum of all the outputs, which gives us a discrete probability distribution[3] over *k* possible classes:

$$softmax(x_i) = \frac{e^{x_i}}{\sum_{j=1}^{k} e^{x_j}}$$

The probabilities in the resulting distribution all sum up to one. This is very useful for interpreting outputs for classification tasks, and so this transformation is usually paired with a probabilistic training objective, such as categorical cross entropy, which is covered in "Diving Deep into Supervised Training".

Example 3-6. Softmax activation

```
Input[0]  import torch.nn as nn
          import torch

          softmax = nn.Softmax(dim=1)
          x_input = torch.randn(1, 3)
          y_output = softmax(x_input)
          print(x_input)
          print(y_output)
          print(torch.sum(y_output, dim=1))
Output[0] tensor([[ 0.5836, -1.3749, -1.1229]])
          tensor([[ 0.7561,  0.1067,  0.1372]])
          tensor([ 1.])
```

In this section, we studied four important activation functions: sigmoid, tanh, ReLU, and softmax. These are but four of the many possible activations that you could use in building neural networks. As we progress through this book, it will become clear which activation functions should be used and where, but a general guide is to simply follow what has worked in the past.

Loss Functions

In Chapter 1, we saw the general supervised machine learning architecture and how loss functions or objective functions help guide the training algorithm to pick the right parameters by looking at the data. Recall that a loss function takes a truth (y) and a prediction ($\hat{y}$) as an input and produces a real-valued score. The higher this score, the worse the model's prediction is. PyTorch implements more loss functions in its `nn` package than we can cover here, but we will review some of the most commonly used loss functions.

Mean Squared Error Loss

For regression problems for which the network's output ($\hat{y}$) and the target (y) are continuous values, one common loss function is the mean squared error (MSE):

$$L_{MSE}(y,\hat{y}) = \frac{1}{n}\sum_{i=1}^{n}(y-\hat{y})^2$$

The MSE is simply the average of the squares of the difference between the predicted and target values. There are several other loss functions that you can use for regression problems, such as mean absolute error (MAE) and root mean squared error (RMSE), but they all involve computing a real-valued distance between the output and target. Example 3-7 shows how you can implement MSE loss using PyTorch.

Example 3-7. MSE loss

```
Input[0]  import torch
          import torch.nn as nn

          mse_loss = nn.MSELoss()
          outputs = torch.randn(3, 5, requires_grad=True)
          targets = torch.randn(3, 5)
          loss = mse_loss(outputs, targets)
          print(loss)
```

```
Output[0] tensor(3.8618)
```

Categorical Cross-Entropy Loss

The categorical cross-entropy loss is typically used in a multiclass classification setting in which the outputs are interpreted as predictions of class membership probabilities. The target (y) is a vector of n elements that represents the true multinomial distribution[4] over all the classes. If only one class is correct, this vector is a one-hot vector. The network's output ($\hat{y}$) is also a vector of n elements but represents the network's prediction of the multinomial distribution. Categorical cross entropy will compare these two vectors (y,$\hat{y}$) to measure the loss:

$$L_{cross\_entropy}\left(y,\hat{y}\right) = -\sum_i y_i \log(\hat{y}_i)$$

Cross-entropy and the expression for it have origins in information theory, but for the purpose of this section it is helpful to consider this as a method to compute how different two distributions are. We want the probability of the correct class to be close to `1`, whereas the other classes have a probability close to `0`.

To correctly use PyTorch's `CrossEntropyLoss()` function, it is important to understand the relationship between the network's outputs, how the loss function is computed, and the kinds of computational constraints that stem from really representing floating-point numbers. Specifically, there are four pieces of information that determine the nuanced relationship between network output and loss function. First, there is a limit to how small or how large a number can be. Second, if input to the exponential function used in the softmax formula is a negative number, the resultant is an exponentially small number, and if it's a positive number, the resultant is an exponentially large number. Next, the network's output is assumed to be the vector just prior to applying the softmax function.[5] Finally, the *log* function is the inverse of the exponential function,[6] and *log(exp(x))* is just equal to *x*. Stemming from these four pieces of information, mathematical simplifications are made assuming the exponential function that is the core of the softmax function and the log function that is used in the cross-entropy computations in order to be more numerically stable and avoid really small or really large numbers. The consequences of these

simplifications are that the network output without the use of a softmax function can be used in conjunction with PyTorch's `CrossEntropyLoss()` to optimize the probability distribution. Then, when the network has been trained, the softmax function can be used to create a probability distribution, as shown in Example 3-8.

Example 3-8. Cross-entropy loss

```
Input[0]  import torch
          import torch.nn as nn

          ce_loss = nn.CrossEntropyLoss()
          outputs = torch.randn(3, 5, requires_grad=True)
          targets = torch.tensor([1, 0, 3], dtype=torch.int64)
          loss = ce_loss(outputs, targets)
          print(loss)

Output[0] tensor(2.7256)
```

In this code example, a vector of random values is first used to simulate network output. Then, the ground truth vector, called `targets`, is created as a vector of integers because PyTorch's implementation of `CrossEntropyLoss()` assumes that each input has one particular class, and each class has a unique index. This is why `targets` has three elements: an index representing the correct class for each input. From this assumption, it performs the computationally more efficient operation of indexing into the model output.[7]

Binary Cross-Entropy Loss

The categorical cross-entropy loss function we saw in the previous section is very useful in classification problems when we have multiple classes. Sometimes, our task involves discriminating between two classes—also known as *binary classification*. For such situations, it is efficient to use the binary cross-entropy (BCE) loss. We look at this loss function in action in "Example: Classifying Sentiment of Restaurant Reviews".

In Example 3-9, we create a binary probability output vector, *probabilities*, using the sigmoid activation function on a random vector that represents the

output of the network. Next, the ground truth is instantiated as a vector of `0`'s and `1`'s.[8] Finally, we compute binary cross-entropy loss using the binary probability vector and the ground truth vector.

Example 3-9. Binary cross-entropy loss

```
Input[0]  bce_loss = nn.BCELoss()
          sigmoid = nn.Sigmoid()
          probabilities = sigmoid(torch.randn(4, 1, requires_grad=True))
          targets = torch.tensor([1, 0, 1, 0],
          dtype=torch.float32).view(4, 1)
          loss = bce_loss(probabilities, targets)
          print(probabilities)
          print(loss)
```

```
Output[0] tensor([[ 0.1625],
                  [ 0.5546],
                  [ 0.6596],
                  [ 0.4284]])
          tensor(0.9003)
```

Diving Deep into Supervised Training

Supervised learning is the problem of learning how to map *observations* to specified *targets* given labeled examples. In this section, we go into more detail. Specifically, we explicitly describe how to use model *predictions* and *a loss function* to do gradient-based optimization of a model's *parameters*. This is an important section because the rest of the book relies on it, so it is worth going through it in detail even if you are somewhat familiar with supervised learning.

Recall from Chapter 1 that supervised learning requires the following: a model, a loss function, training data, and an optimization algorithm. The training data for supervised learning is pairs of observations and targets; the model computes predictions from the observations, and the loss measures the error of the predictions as compared to the targets. The goal of the training is to use the gradient-based optimization algorithm to adjust the model's parameters so that the losses are as low as possible.

In the remainder of this section, we discuss a classic toy problem: classifying two-dimensional points into one of two classes. Intuitively, this

means learning a single line, called a *decision boundary* or *hyperplane*, to discriminate the points of one class from the other. We step through and describe the data construction, choosing the model, selecting a loss function, setting up the optimization algorithm, and, finally, running it all together.

Constructing Toy Data

In machine learning, it is a common practice to create synthetic data with well-understood properties when trying to understand an algorithm. For this section, we use synthetic data for the task of classifying two-dimensional points into one of two classes. To construct the data, we sample[9] the points from two different parts of the xy-plane, creating an easy-to-learn situation for the model. Samples are shown in the plot depicted in Figure 3-2. The goal of the model is to classify the stars (⋆) as one class, and the circles (◯) as another class. This is visualized on the righthand side, where everything above the line is classified differently than everything below the line. The code for generating the data is in the function named `get_toy_data()` in the Python notebook that accompanies this chapter.

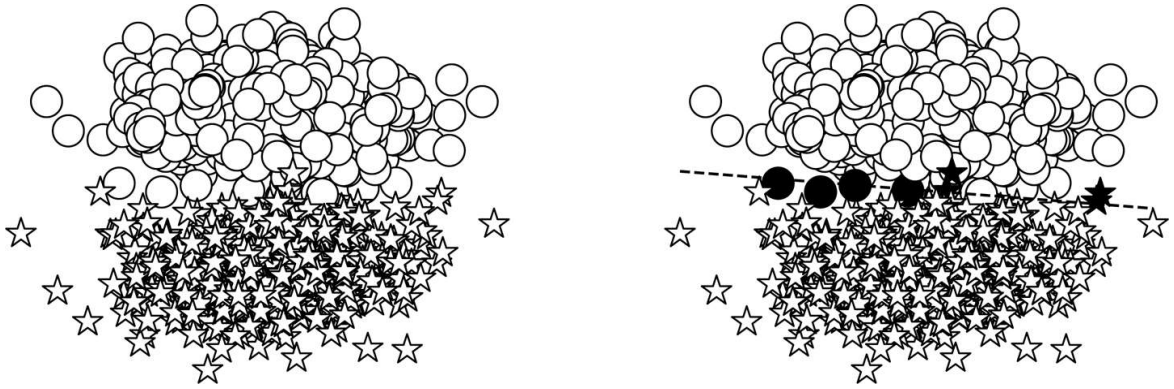


Figure 3-2. Creating a toy dataset that's linearly separable. The dataset is a sampling from two normal distributions, one for each class; the classification task becomes one of distinguishing whether a data point belongs to one distribution or the other.

Choosing a model

The model we use here is the one introduced at the beginning of the chapter: the perceptron. The perceptron is flexible in that it allows for any input size. In a typical modeling situation, the input size is determined by

the task and data. In this toy example, the input size is 2 because we explicitly constructed the data to be in a two-dimensional plane. For this two-class problem, we assign a numeric indices to the classes: `0` and `1`. The mapping of the string labels ⋆ and ◯ to the class indexes is arbitrary as long as it is consistent throughout data preprocessing, training, evaluation, and testing. An important, additional property of this model is the nature of its output. Due to the perceptron's activation function being a sigmoid, the output of the perceptron is the probability of the data point (x) being class `1`; that is, $P(y = \texttt{1} \mid x)$.

Converting the probabilities to discrete classes

For the binary classification problem, we can convert the output probability into two discrete classes by imposing a decision boundary, δ. If the predicted probability $P(y = \texttt{1} \mid x) > \delta$, the predicted class is `1`, else the class is `0`. Typically, this decision boundary is set to be `0.5`, but in practice, you might need to tune this hyperparameter (using an evaluation dataset) to achieve a desired precision in classification.

Choosing a loss function

After you have prepared the data and selected a model architecture, there are two other vital components to choose in supervised training: a loss function and an optimizer. For situations in which the model's output is a probability, the most appropriate family of loss functions are cross entropy–based losses. For this toy data example, because the model is producing binary outcomes, we specifically use the BCE loss.

Choosing an optimizer

The final choice point in this simplified supervised training example is the optimizer. While the model produces predictions and the loss function measures the error between predictions and targets, the optimizer updates the weights of the model using the error signal. In its simplest form, there is a single hyperparameter that controls the update behavior of the optimizer. This hyperparameter, called a *learning rate*, controls how much impact the error signal has on updating the weights. Learning rate is a critical

hyperparameter, and you should try several different learning rates and compare them. Large learning rates will cause bigger changes to the parameters and can affect convergence. Too-small learning rates can result in very little progress during training.

The PyTorch library offers several choices for an optimizer. Stochastic gradient descent (SGD) is a classic algorithm of choice, but for difficult optimization problems, SGD has convergence issues, often leading to poorer models. The current preferred alternative are adaptive optimizers, such as Adagrad or Adam, which use information about updates over time.[10] In the following example we use Adam, but it is always worth looking at several optimizers. With Adam, the default learning rate is `0.001`. With hyperparameters such as learning rate, it's always recommended to use the default values first, unless you have a recipe from a paper calling for a specific value.

Example 3-10. Instantiating the Adam optimizer

```
Input[0] import torch.nn as nn
         import torch.optim as optim

         input_dim = 2
         lr = 0.001

         perceptron = Perceptron(input_dim=input_dim)
         bce_loss = nn.BCELoss()
         optimizer = optim.Adam(params=perceptron.parameters(), lr=lr)
```

Putting It Together: Gradient-Based Supervised Learning

Learning begins with computing the loss; that is, how far off the model predictions are from the target. The gradient of the loss function, in turn, becomes a signal for "how much" the parameters should change. The gradient for each parameter represents instantaneous rate of change in the loss value given the parameter. Effectively, this means that you can know how much each parameter contributed to the loss function. Intuitively, this is a slope and you can imagine each parameter is standing on its own hill and wants to take a step up or down the hill. In its most minimal form, all that is involved with gradient-based model training is iteratively updating

each parameter with the gradient of the loss function with respect to that parameter.

Let's take a look at how this gradient-stepping algorithm looks. First, any bookkeeping information, such as gradients, currently stored inside the model (`perceptron`) object is cleared with a function named `zero_grad()`. Then, the model computes outputs (`y_pred`) given the input data (`x_data`). Next, the loss is computed by comparing model outputs (`y_pred`) to intended targets (`y_target`). This is the supervised part of the supervised training signal. The PyTorch loss object (`criterion`) has a function named `backward()` that iteratively propagates the loss backward through the computational graph and notifies each parameter of its gradient. Finally, the optimizer (`opt`) instructs the parameters how to update their values knowing the gradient with a function named `step()`.

The entire training dataset is partitioned into *batches*. Each iteration of the gradient step is performed on a batch of data. A hyperparameter named `batch_size` specifies the size of the batches. Because the training dataset is fixed, increasing the batch size decreases the number of batches.

> **NOTE**
>
> In the literature, and also in this book, the term *minibatch* is used interchangeably with batch to highlight that each of the batches is significantly smaller than the size of the training data; for example, the training data size could be in the millions, whereas the minibatch could be just a few hundred in size.

After a number of batches (typically, the number of batches that are in a finite-sized dataset), the training loop has completed an *epoch*. An epoch is a complete training iteration. If the number of batches per epoch is the same as the number of batches in a dataset, then an epoch is a complete iteration over a dataset. Models are trained for a certain number of epochs. The number of epochs to train is not trivial to select, but there are methods for determining when to stop, which we discuss shortly. As Example 3-11

illustrates, the supervised training loop is thus a nested loop: an inner loop over a dataset or a set number of batches, and an outer loop, which repeats the inner loop over a fixed number of epochs or other termination criteria.

Example 3-11. A supervised training loop for a perceptron and binary classification

```
# each epoch is a complete pass over the training data
for epoch_i in range(n_epochs):
    # the inner loop is over the batches in the dataset
    for batch_i in range(n_batches):

        # Step 0: Get the data
        x_data, y_target = get_toy_data(batch_size)

        # Step 1: Clear the gradients
        perceptron.zero_grad()

        # Step 2: Compute the forward pass of the model
        y_pred = perceptron(x_data, apply_sigmoid=True)

        # Step 3: Compute the loss value that we wish to optimize
        loss = bce_loss(y_pred, y_target)

        # Step 4: Propagate the loss signal backward
        loss.backward()

        # Step 5: Trigger the optimizer to perform one update
        optimizer.step()
```

Auxiliary Training Concepts

The core idea of supervised gradient-based learning is simple: define a model, compute outputs, use a loss function to compute gradients, and apply an optimization algorithm to update model parameters with the gradient. However, there are several auxiliary concepts that are important to the training process. We cover a few of them in this section.

Correctly Measuring Model Performance: Evaluation Metrics

The most important component outside of the core supervised training loop is an objective measure of performance using data on which the model has never trained. Models are evaluated using one or more *evaluation metrics*. In natural language processing, there are multiple such metrics. The most common, and the one we will use in this chapter, is *accuracy*. Accuracy is simply the fraction of the predictions that were correct on a dataset unseen during training.

Correctly Measuring Model Performance: Splitting the Dataset

It is important to always keep in mind that the final goal is to *generalize* well to the true distribution of data. What do we mean by that? There exists a distribution of data that exists globally assuming we were able see an infinite amount of data ("*true/unseen distribution*"). Obviously, we cannot do that. Instead, we make do with a finite sample that we call the training data. We observe a distribution of data in the finite sample that's an approximation or an incomplete picture of the true distribution. A model is said to have *generalized better* than another model if it not only reduces the error on samples seen in the training data, but also on the samples from the unseen distribution. As the model works toward lowering its loss on the training data, it can "overfit" and adapt to idiosyncrasies that aren't actually part of the true data distribution.

To accomplish this goal of generalizing well, it is standard practice to either split a dataset into three randomly sampled partitions (called the *training*, *validation*, and *test* datasets) or do *k-fold cross validation*. Splitting into three partitions is the simpler of the two methods because it only requires a single computation. You should take precautions to make sure the distribution of classes remains the same between each of the three splits, however. In other words, it is good practice to aggregate the dataset by class label and then randomly split each set separated by class label into the training, validation, and test datasets. A common split percentage is to

reserve 70% for training, 15% for validation, and 15% for testing. This is not a hardcoded convention, though.

In some cases, a predefined training, validation, and test split might exist; this is common in datasets for benchmarking tasks. In such cases, it is important to use training data only for updating model parameters, use validation data for measuring model performance at the end of every epoch, and use test data only once, after all modeling choices are explored and the final results need to be reported. This last part is extremely important because the more the machine learning engineer peeks at the model performance on a test dataset, the more they are biased toward choices which perform better on the test set. When this happens, it is impossible to know how the model will perform on unseen data without gathering more data.

Model evaluation with *k*-fold cross validation is very similar to evaluation with predefined splits, but is preceded by an extra step of splitting the entire dataset into *k* equally sized "folds." One of the folds is reserved for evaluation, and the remaining *k-1* folds for training. This is iteratively repeated by swapping out the fold used for evaluation. Because there are *k* folds, each fold gets a chance to become an evaluation fold, resulting in *k* accuracy values. The final reported accuracy is simply the average with standard deviation. *k*–fold evaluation is computationally expensive but extremely necessary for smaller datasets, for which the wrong split can lead to either too much optimism (because the testing data wound up being too easy) or too much pessimism (because the testing data wound up being too hard).

Knowing When to Stop Training

The example earlier trained the model for a fixed number of epochs. Although this is the simplest approach, it is arbitrary and unnecessary. One key function of correctly measuring model performance is to use that measurement to determine when training should stop. The most common method is to use a heuristic called *early stopping*. Early stopping works by keeping track of the performance on the validation dataset from epoch to epoch and noticing when the performance no longer improves. Then, if the

performance continues to not improve, the training is terminated. The number of epochs to wait before terminating the training is referred to as the *patience*. In general, the point at which a model stops improving on some dataset is said to be when the model has *converged*. In practice, we rarely wait for a model to completely converge because convergence is time-consuming, and it can lead to overfitting.

Finding the Right Hyperparameters

We learned earlier that a parameter (or weight) takes real values adjusted by an optimizer with respect to a fixed subset of training data called a minibatch. A *hyperparameter* is any model setting that affects the number of parameters in the model and values taken by the parameters. There are many different choices that go into determining how the model is trained. These include choosing a loss function; the optimizer; learning rate(s) for the optimizer, as layer sizes (covered in Chapter 4); patience for early stopping; and various regularization decisions (also covered in Chapter 4). It is important to be mindful that these decisions can have large effects on whether a model converges and its performance, and you should explore the various choice points systematically.

Regularization

One of the most important concepts in deep learning (and machine learning, in general) is *regularization*. The concept of regularization comes from numerical optimization theory. Recall that most machine learning algorithms are optimizing the loss function to find the most likely values of the parameters (or "the model") that explains the observations (i.e., produces the least amount of loss). For most datasets and tasks, there could be multiple solutions (possible models) to this optimization problem. So which one should we (or the optimizer) pick? To develop an intuitive understanding, consider Figure 3-3 for the task of fitting a curve through a set of points.

Both curves "fit" the points, but which one is an unlikely explanation? By appealing to Occam's razor, we intuit that the simpler explanation is better than the complex one. This smoothness constraint in machine learning is

called *L2 regularization*. In PyTorch, you can control this by setting the `weight_decay` parameter in the optimizer. The larger the `weight_decay` value, the more likely it is that the optimizer will select the smoother explanation (that is, the stronger is the L2 regularization).

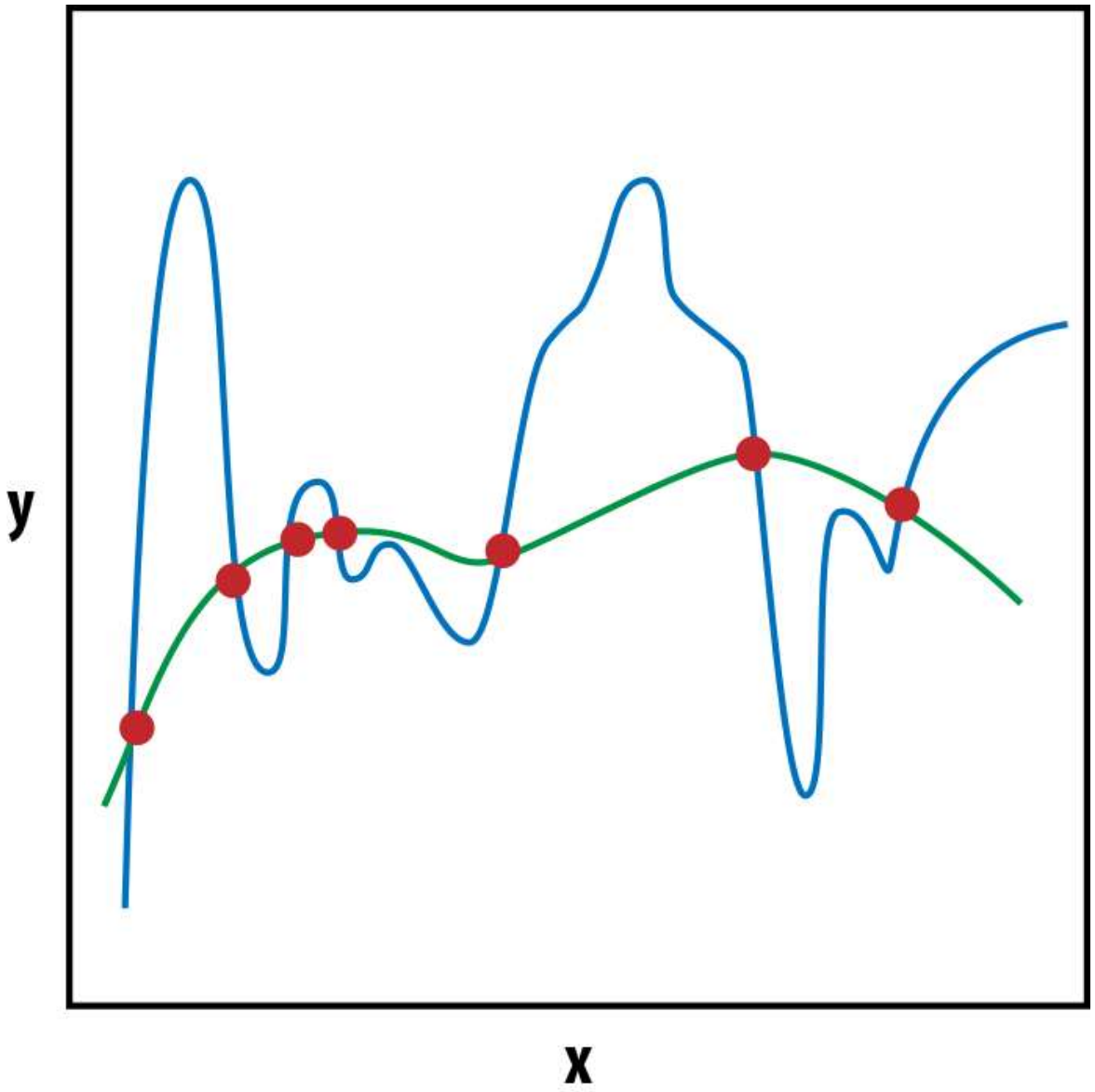


Figure 3-3. Both curves "fit" the points, but one of them seems more reasonable than the other—regularization helps us to select this more reasonable explanation (courtesy of Wikipedia).

In addition to L2, another popular regularization is *L1 regularization*. L1 is usually used to encourage sparser solutions; in other words, where most of the model parameter values are close to zero. In Chapter 4, we will look at

another structural regularization technique, called *dropout*. The topic of model regularization is an active area of research and PyTorch is a flexible framework for implementing custom regularizers.

Example: Classifying Sentiment of Restaurant Reviews

In the previous section, we dove deep into supervised training with a toy example and illustrated many fundamental concepts. In this section we repeat that exercise, but this time with a real-world task and dataset: to classify whether restaurant reviews on Yelp are positive or negative using a perceptron and supervised training. Because this is the first full NLP example in this book, we will describe the assisting data structures and training routine in excruciating detail. The examples in later chapters will follow very similar patterns, so we encourage you to carefully follow along with this section and refer back to it as needed for a refresher.[11]

At the start of each example in this book, we describe the dataset that we are using. In this example we use the Yelp dataset, which pairs reviews with their sentiment labels (positive or negative). We additionally describe a couple of dataset manipulation steps we took to clean and partition it into training, validation, and test sets.

After understanding the dataset, you will see a pattern defining three assisting classes that is repeated throughout this book and is used to transform text data into a vectorized form: the `Vocabulary`, the `Vectorizer`, and PyTorch's `DataLoader`. The `Vocabulary` coordinates the integer-to-token mappings that we discussed in "Observation and Target Encoding". We use a `Vocabulary` both for mapping the text tokens to integers and for mapping the class labels to integers. Next, the `Vectorizer` encapsulates the vocabularies and is responsible for ingesting string data, like a review's text, and converting it to numerical vectors that will be used in the training routine. We use the final assisting class, PyTorch's `DataLoader`, to group and collate the individual vectorized data points into minibatches.

The following section describes the perceptron classifier and its training routine. The training routine mostly remains the same for every example in this book, but we discuss it in more detail in this section, so again, we encourage you to use this example as a reference for future training routines. We conclude the example by discussing the results and taking a peek under the hood to see what the model learned.

The Yelp Review Dataset

In 2015, Yelp held a contest in which it asked participants to predict the rating of a restaurant given its review. Zhang, Zhao, and Lecun (2015) simplified the dataset by converting the 1- and 2-star ratings into a "negative" sentiment class and the 3- and 4-star ratings into a "positive" sentiment class, and split it into 560,000 training samples and 38,000 testing samples. In this example we use the simplified Yelp dataset, with two minor differences. In the remainder of this section, we describe the process by which we minimally clean the data and derive our final dataset. Then, we outline the implementation that utilizes PyTorch's `Dataset` class.

The first of the differences mentioned is that we use a "light" version of the dataset, which is derived by selecting 10% of the training samples as the full dataset.[12] This has two consequences. First, using a small dataset makes the training–testing loop fast, so we can experiment quickly. Second, it produces a model with lower accuracy than would be achieved by using all of the data. This low accuracy is usually not a major issue, because you can retrain with the entire dataset using the knowledge gained from the smaller subset. This is a very useful trick in training deep learning models, where the amount of training data in many situations can be enormous.

From this smaller subset, we split the dataset into three partitions: one for training, one for validation, and one for testing. Although the original dataset has only two partitions, it's important to have a validation set. In machine learning, you will often train a model on the training partition of a dataset and require a held-out partition for evaluating how well the model did. If model decisions are based on that held-out portion, the model will inevitably be biased toward performing better on the held-out portion. Because measuring incremental progress is vital, the solution to this

problem is to have a third partition, which is used for evaluation as little as possible.

To summarize, you should use the training partition of a dataset to derive model parameters, the validation partition of a dataset for selecting among hyperparameters (making modeling decisions), and the testing partition of the dataset for final evaluation and reporting.[13] In Example 3-12, we show how we split the dataset. Note that the random seed is set to a static number and that we first aggregate by class label to guarantee the class distribution remains the same.

Example 3-12. Creating training, validation, and testing splits

```
# Split the subset by rating to create new train, val, and test
splits
by_rating = collections.defaultdict(list)
for _, row in review_subset.iterrows():
    by_rating[row.rating].append(row.to_dict())

# Create split data
final_list = []
np.random.seed(args.seed)

for _, item_list in sorted(by_rating.items()):
    np.random.shuffle(item_list)

    n_total = len(item_list)
    n_train = int(args.train_proportion * n_total)
    n_val = int(args.val_proportion * n_total)
    n_test = int(args.test_proportion * n_total)

    # Give data point a split attribute
    for item in item_list[:n_train]:
        item['split'] = 'train'

    for item in item_list[n_train:n_train+n_val]:
        item['split'] = 'val'

    for item in item_list[n_train+n_val:n_train+n_val+n_test]:
        item['split'] = 'test'
```

```
    # Add to final list
    final_list.extend(item_list)

final_reviews = pd.DataFrame(final_list)
```

In addition to creating a subset that has three partitions for training, validation, and testing, we also minimally clean the data by adding whitespace around punctuation symbols and removing extraneous symbols that aren't punctuation for all the splits, as shown in Example 3-13.[14]

Example 3-13. Minimally cleaning the data

```
def preprocess_text(text):
    text = text.lower()
    text = re.sub(r"([.,!?])", r" \1 ", text)
    text = re.sub(r"[^a-zA-Z.,!?]+", r" ", text)
    return text

final_reviews.review = final_reviews.review.apply(preprocess_text)
```

Understanding PyTorch's Dataset Representation

The `ReviewDataset` class presented in Example 3-14 assumes that the dataset that has been minimally cleaned and split into three partitions. In particular, the dataset assumes that it can split reviews based on whitespace in order to get the list of tokens in a review.[15] Further, it assumes that the data has an annotation for which split it belongs to. It is important to notice that we indicate the entry point method for this dataset class using Python's `classmethod` decorator. We follow this pattern throughout the book.

PyTorch provides an abstraction for the dataset by providing a `Dataset` class. The `Dataset` class is an abstract iterator. When using PyTorch with a new dataset, you must first subclass (or inherit from) the `Dataset` class and implement these `__getitem__()` and `__len__()` methods. For this example, we create a `ReviewDataset` class that inherits from PyTorch's `Dataset` class and implements the two methods: `__getitem__` and `__len__`. This creates a conceptual pact that allows various PyTorch utilities to work with our dataset. We cover one of these utilities, in particular the `DataLoader`, in the next section. The implementation that

follows relies heavily on a class called `ReviewVectorizer`. We describe the `ReviewVectorizer` in the next section, but intuitively you can picture it as a class that handles the conversion from review text to a vector of numbers representing the review. Only through some vectorization step can a neural network interact with text data. The overall design pattern is to implement a dataset class that handles the vectorization logic for one data point. Then, PyTorch's `DataLoader` (also described in the next section) will create minibatches by sampling and collating from the dataset.

Example 3-14. A PyTorch Dataset class for the Yelp Review dataset

```
from torch.utils.data import Dataset

class ReviewDataset(Dataset):
    def __init__(self, review_df, vectorizer):
        """
        Args:
            review_df (pandas.DataFrame): the dataset
            vectorizer (ReviewVectorizer): vectorizer instantiated
from dataset
        """
        self.review_df = review_df
        self._vectorizer = vectorizer

        self.train_df = self.review_df[self.review_df.split=='train']
        self.train_size = len(self.train_df)

        self.val_df = self.review_df[self.review_df.split=='val']
        self.validation_size = len(self.val_df)

        self.test_df = self.review_df[self.review_df.split=='test']
        self.test_size = len(self.test_df)

        self._lookup_dict = {'train': (self.train_df,
self.train_size),
                             'val': (self.val_df,
self.validation_size),
                             'test': (self.test_df, self.test_size)}

        self.set_split('train')
```

```
    @classmethod
    def load_dataset_and_make_vectorizer(cls, review_csv):
        """Load dataset and make a new vectorizer from scratch

        Args:
            review_csv (str): location of the dataset
        Returns:
            an instance of ReviewDataset
        """
        review_df = pd.read_csv(review_csv)
        return cls(review_df,
ReviewVectorizer.from_dataframe(review_df))

    def get_vectorizer(self):
        """ returns the vectorizer """
        return self._vectorizer

    def set_split(self, split="train"):
        """ selects the splits in the dataset using a column in the
dataframe

        Args:
            split (str): one of "train", "val", or "test"
        """
        self._target_split = split
        self._target_df, self._target_size = self._lookup_dict[split]

    def __len__(self):
        return self._target_size

    def __getitem__(self, index):
        """the primary entry point method for PyTorch datasets

        Args:
            index (int): the index to the data point
        Returns:
            a dict of the data point's features (x_data) and label
(y_target)
        """
        row = self._target_df.iloc[index]
```

```
        review_vector = \
            self._vectorizer.vectorize(row.review)

        rating_index = \
            self._vectorizer.rating_vocab.lookup_token(row.rating)

        return {'x_data': review_vector,
                'y_target': rating_index}

    def get_num_batches(self, batch_size):
        """Given a batch size, return the number of batches in the
dataset

        Args:
            batch_size (int)
        Returns:
            number of batches in the dataset
        """
        return len(self) // batch_size
```

The Vocabulary, the Vectorizer, and the DataLoader

The `Vocabulary`, the `Vectorizer`, and the `DataLoader` are three classes that we use in nearly every example in this book to perform a crucial pipeline: converting text inputs to vectorized minibatches. The pipeline starts with preprocessed text; each data point is a collection of tokens. In this example, the tokens happen to be words, but as you will see in Chapter 4 and Chapter 6, tokens can also be characters. The three classes presented in the following subsections are responsible for mapping each token to an integer, applying this mapping to each data point to create a vectorized form, and then grouping the vectorized data points into a minibatch for the model.

Vocabulary

The first stage in going from text to vectorized minibatch is to map each token to a numerical version of itself. The standard methodology is to have a bijection—a mapping that can be reversed—between the tokens and

integers. In Python, this is simply two dictionaries. We encapsulate this bijection into a `Vocabulary` class, shown in Example 3-15. The `Vocabulary` class not only manages this bijection—allowing the user to add new tokens and have the index autoincrement—but also handles a special token called UNK,[16] which stands for "unknown." By using the `UNK` token, we can handle tokens at test time that were never seen in training (for instance, you might encounter words that were not encountered in the training dataset). As we will see in the `Vectorizer` next, we will even explicitly restrict infrequent tokens from our `Vocabulary` so that there are `UNK` tokens in our training routine. This is essential in limiting the memory used by the `Vocabulary` class.[17] The expected behavior is that `add_token()` is called to add new tokens to the `Vocabulary`, `lookup_token()` when retrieving the index for a token, and `lookup_index()` when retrieving the token corresponding to a specific index.

Example 3-15. The Vocabulary class maintains token to integer mapping needed for the rest of the machine learning pipeline

```
class Vocabulary(object):
    """Class to process text and extract Vocabulary for mapping"""

    def __init__(self, token_to_idx=None, add_unk=True, unk_token="
<UNK>"):
        """
        Args:
            token_to_idx (dict): a pre-existing map of tokens to
indices
            add_unk (bool): a flag that indicates whether to add the
UNK token
            unk_token (str): the UNK token to add into the Vocabulary
        """

        if token_to_idx is None:
            token_to_idx = {}
        self._token_to_idx = token_to_idx

        self._idx_to_token = {idx: token
                              for token, idx in
```

```
self._token_to_idx.items()}

        self._add_unk = add_unk
        self._unk_token = unk_token

        self.unk_index = -1
        if add_unk:
            self.unk_index = self.add_token(unk_token)


    def to_serializable(self):
        """ returns a dictionary that can be serialized """
        return {'token_to_idx': self._token_to_idx,
                'add_unk': self._add_unk,
                'unk_token': self._unk_token}

    @classmethod
    def from_serializable(cls, contents):
        """ instantiates the Vocabulary from a serialized dictionary
"""
        return cls(**contents)

    def add_token(self, token):
        """Update mapping dicts based on the token.

        Args:
            token (str): the item to add into the Vocabulary
        Returns:
            index (int): the integer corresponding to the token
        """
        if token in self._token_to_idx:
            index = self._token_to_idx[token]
        else:
            index = len(self._token_to_idx)
            self._token_to_idx[token] = index
            self._idx_to_token[index] = token
        return index

    def lookup_token(self, token):
        """Retrieve the index associated with the token
          or the UNK index if token isn't present.
```

```
        Args:
            token (str): the token to look up
        Returns:
            index (int): the index corresponding to the token
        Notes:
            `unk_index` needs to be >=0 (having been added into the
Vocabulary)
              for the UNK functionality
        """
        if self.add_unk:
            return self._token_to_idx.get(token, self.unk_index)
        else:
            return self._token_to_idx[token]

    def lookup_index(self, index):
        """Return the token associated with the index

        Args:
            index (int): the index to look up
        Returns:
            token (str): the token corresponding to the index
        Raises:
            KeyError: if the index is not in the Vocabulary
        """
        if index not in self._idx_to_token:
            raise KeyError("the index (%d) is not in the Vocabulary"
% index)
        return self._idx_to_token[index]

    def __str__(self):
        return "<Vocabulary(size=%d)>" % len(self)

    def __len__(self):
        return len(self._token_to_idx)
```

Vectorizer

The second stage of going from a text dataset to a vectorized minibatch is to iterate through the tokens of an input data point and convert each token to its integer form. The result of this iteration should be a vector. Because this

vector will be combined with vectors from other data points, there is a constraint that the vectors produced by the `Vectorizer` should always have the same length.

To accomplish these goals, the `Vectorizer` class encapsulates the review `Vocabulary`, which maps words in the review to integers. In Example 3-16, the `Vectorizer` utilizes Python's `@classmethod` decorator for the method `from_dataframe()` to indicate an entry point to instantiating the `Vectorizer`. The method `from_dataframe()` iterates over the rows of a Pandas DataFrame with two goals. The first goal is to count the frequency of all tokens present in the dataset. The second goal is to create a `Vocabulary` that only uses tokens that are as frequent as a provided keyword argument to the method, `cutoff`. Effectively, this method is finding all words that occur at least `cutoff` times and adding them to the `Vocabulary`. Because the `UNK` token is also added to the `Vocabulary`, any words that are not added will have the `unk_index` when the `Vocabulary`'s `lookup_token()` method is called.

The method `vectorize()` encapsulates the core functionality of the `Vectorizer`. It takes as an argument a string representing a review and returns a vectorized representation of the review. In this example, we use the collapsed one-hot representation that we introduced in Chapter 1. This representation creates a binary vector—a vector of `1`s and `0`s—that has a length equal to the size of the `Vocabulary`. The binary vector has `1` in the locations that correspond to the words in the review. Note that this representation has some limitations. The first is that it is sparse—the number of unique words in the review will always be far less than the number of unique words in the `Vocabulary`. The second is that it discards the order in which the words appeared in the review (the "bag of words" approach). In the subsequent chapters, you will see other methods that don't have these limitations.

Example 3-16. The Vectorizer class converts text to numeric vectors

```
class ReviewVectorizer(object):
    """ The Vectorizer which coordinates the Vocabularies and puts
them to use"""
```

```
def __init__(self, review_vocab, rating_vocab):
    """
    Args:
        review_vocab (Vocabulary): maps words to integers
        rating_vocab (Vocabulary): maps class labels to integers
    """
    self.review_vocab = review_vocab
    self.rating_vocab = rating_vocab

def vectorize(self, review):
    """Create a collapsed one-hit vector for the review

    Args:
        review (str): the review
    Returns:
        one_hot (np.ndarray): the collapsed one-hot encoding
    """
    one_hot = np.zeros(len(self.review_vocab), dtype=np.float32)

    for token in review.split(" "):
        if token not in string.punctuation:
            one_hot[self.review_vocab.lookup_token(token)] = 1

    return one_hot

@classmethod
def from_dataframe(cls, review_df, cutoff=25):
    """Instantiate the vectorizer from the dataset dataframe

    Args:
        review_df (pandas.DataFrame): the review dataset
        cutoff (int): the parameter for frequency-based filtering
    Returns:
        an instance of the ReviewVectorizer
    """
    review_vocab = Vocabulary(add_unk=True)
    rating_vocab = Vocabulary(add_unk=False)

    # Add ratings
    for rating in sorted(set(review_df.rating)):
        rating_vocab.add_token(rating)
```

```
        # Add top words if count > provided count
        word_counts = Counter()
        for review in review_df.review:
            for word in review.split(" "):
                if word not in string.punctuation:
                    word_counts[word] += 1

        for word, count in word_counts.items():
            if count > cutoff:
                review_vocab.add_token(word)

        return cls(review_vocab, rating_vocab)

    @classmethod
    def from_serializable(cls, contents):
        """Intantiate a ReviewVectorizer from a serializable
dictionary

        Args:
            contents (dict): the serializable dictionary
        Returns:
            an instance of the ReviewVectorizer class
        """
        review_vocab =
Vocabulary.from_serializable(contents['review_vocab'])
        rating_vocab
=  Vocabulary.from_serializable(contents['rating_vocab'])

        return cls(review_vocab=review_vocab,
rating_vocab=rating_vocab)

    def to_serializable(self):
        """Create the serializable dictionary for caching

        Returns:
            contents (dict): the serializable dictionary
        """
        return {'review_vocab': self.review_vocab.to_serializable(),
                'rating_vocab': self.rating_vocab.to_serializable()}
```

DataLoader

The final stage of the text-to-vectorized-minibatch pipeline is to actually group the vectorized data points. Because grouping into minibatches is a vital part of training neural networks, PyTorch provides a built-in class called `DataLoader` for coordinating the process. The `DataLoader` class is instantiated by providing a PyTorch `Dataset` (such as the `ReviewDataset` defined for this example), a `batch_size`, and a handful of other keyword arguments. The resulting object is a Python iterator that groups and collates the data points provided in the `Dataset`.[18] In Example 3-17, we wrap the `DataLoader` in a `generate_batches()` function, which is a generator to conveniently switch the data between the CPU and the GPU.

Example 3-17. Generating minibatches from a dataset

```
def generate_batches(dataset, batch_size, shuffle=True,
                     drop_last=True, device="cpu"):
    """
    A generator function which wraps the PyTorch DataLoader. It will
      ensure each tensor is on the write device location.
    """
    dataloader = DataLoader(dataset=dataset, batch_size=batch_size,
                            shuffle=shuffle, drop_last=drop_last)

    for data_dict in dataloader:
        out_data_dict = {}
        for name, tensor in data_dict.items():
            out_data_dict[name] = data_dict[name].to(device)
        yield out_data_dict
```

A Perceptron Classifier

The model we use in this example is a reimplementation of the `Perceptron` classifier we showed at the beginning of the chapter. The `ReviewClassifier` inherits from PyTorch's `Module` and creates a single `Linear` layer with a single output. Because this is a binary classification setting (negative or positive review), this is an appropriate setup. The sigmoid function is used as the final nonlinearity.

We parameterize the `forward()` method to allow for the sigmoid function to be optionally applied. To understand why, it is important to first point out that in a binary classification task, binary cross-entropy loss (`torch.nn.BCELoss()`) is the most appropriate loss function. It is mathematically formulated for binary probabilities. However, there are numerical stability issues with applying a sigmoid and then using this loss function. To provide its users with shortcuts that are more numerically stable, PyTorch provides `BCEWithLogitsLoss()`. To use this loss function, the output should not have the sigmoid function applied. Therefore, by default, we do not apply the sigmoid. However, in the case that the user of the classifier would like a probability value, the sigmoid is required, and it is left as an option. We see an example of it being used in this way in the results section in Example 3-18.

Example 3-18. A perceptron classifier for classifying Yelp reviews

```
import torch.nn as nn
import torch.nn.functional as F

class ReviewClassifier(nn.Module):
    """ a simple perceptron-based classifier """
    def __init__(self, num_features):
        """
        Args:
            num_features (int): the size of the input feature vector
        """
        super(ReviewClassifier, self).__init__()
        self.fc1 = nn.Linear(in_features=num_features,
                             out_features=1)

    def forward(self, x_in, apply_sigmoid=False):
        """The forward pass of the classifier

        Args:
            x_in (torch.Tensor): an input data tensor
                x_in.shape should be (batch, num_features)
            apply_sigmoid (bool): a flag for the sigmoid activation
                should be false if used with the cross-entropy losses
        Returns:
            the resulting tensor. tensor.shape should be (batch,).
```

```
        """
        y_out = self.fc1(x_in).squeeze()
        if apply_sigmoid:
            y_out = F.sigmoid(y_out)
        return y_out
```

The Training Routine

In this section, we outline the components of the training routine and how they come together with the dataset and model to adjust the model parameters and increase its performance. At its core, the training routine is responsible for instantiating the model, iterating over the dataset, computing the output of the model when given the data as input, computing the loss (how wrong the model is), and updating the model proportional to the loss. Although this may seem like a lot of details to manage, there are not many places to change the training routine, and as such it will become habitual in your deep learning development process. To aid in management of the higher-level decisions, we make use of an `args` object to centrally coordinate all decision points, which you can see in Example 3-19.[19]

Example 3-19. Hyperparameters and program options for the perceptron-based Yelp review classifier

```
from argparse import Namespace

args = Namespace(
    # Data and path information
    frequency_cutoff=25,
    model_state_file='model.pth',
    review_csv='data/yelp/reviews_with_splits_lite.csv',
    save_dir='model_storage/ch3/yelp/',
    vectorizer_file='vectorizer.json',
    # No model hyperparameters
    # Training hyperparameters
    batch_size=128,
    early_stopping_criteria=5,
    learning_rate=0.001,
    num_epochs=100,
    seed=1337,
```

```
    # Runtime options omitted for space
)
```

In the remainder of this section, we first describe the *training state*, a small dictionary that we use to track information about the training process. This dictionary will grow as you track more details about the training routine, and you can systematize it if you choose to do so, but the dictionary presented in our next example is the basic set of information you will be tracking in during model training. After describing the training state, we outline the set of objects that are instantiated for model training to be executed. This includes the model itself, the dataset, the optimizer, and the loss function. In other examples and in the supplementary material, we include additional components, but we do not list them in the text for simplicity. Finally, we wrap up this section with the training loop itself and demonstrate the standard PyTorch optimization pattern.

Setting the stage for the training to begin

Example 3-20 shows the training components that we instantiate for this example. The first item is the initial training state. The function accepts the `args` object as an argument so that the training state can handle complex information, but in the text in this book, we do not show any of these complexities. We refer you to the supplementary material to see what additional things you can use in the training state. The minimal set shown here includes the epoch index and lists for the training loss, training accuracy, validation loss, and validation accuracy. It also includes two fields for the test loss and test accuracy.

The next two items to be instantiated are the dataset and the model. In this example, and in the examples in the remainder of the book, we design the datasets to be responsible for instantiating the vectorizers. In the supplementary material, the dataset instantiation is nested in an `if` statement that allows either the loading of previously instantiated vectorizers or a new instantiation that will also save the vectorizer to disk. The model is importantly moved to the correct device by coordinating with the wishes of the user (through `args.cuda`) and a conditional that checks whether a GPU device is indeed available. The target device is used in the

`generate_batches()` function call in the core training loop so that the data and model will be in the same device location.

The last two items in the initial instantiation are the loss function and the optimizer. The loss function used in this example is `BCEWithLogitsLoss()`. (As mentioned in "A Perceptron Classifier", the most appropriate loss function for binary classification is binary cross-entropy loss, and it is more numerically stable to pair the `BCEWithLogitsLoss()` function with a model that doesn't apply the sigmoid function to the output than to pair the `BCELoss()` function with a model that does apply the sigmoid function to the output.) The optimizer we use is the Adam optimizer. In general, Adam is highly competitive with other optimizers, and as of this writing there is no compelling evidence to use any other optimizer over Adam. We do encourage you to verify this for yourself by trying other optimizers and noting the performance.

Example 3-20. Instantiating the dataset, model, loss, optimizer, and training state

```
import torch.optim as optim

def make_train_state(args):
    return {'epoch_index': 0,
            'train_loss': [],
            'train_acc': [],
            'val_loss': [],
            'val_acc': [],
            'test_loss': -1,
            'test_acc': -1}
train_state = make_train_state(args)

if not torch.cuda.is_available():
    args.cuda = False
args.device = torch.device("cuda" if args.cuda else "cpu")

# dataset and vectorizer
dataset =
ReviewDataset.load_dataset_and_make_vectorizer(args.review_csv)
vectorizer = dataset.get_vectorizer()
```

```
# model
classifier =
ReviewClassifier(num_features=len(vectorizer.review_vocab))
classifier = classifier.to(args.device)

# loss and optimizer
loss_func = nn.BCEWithLogitsLoss()
optimizer = optim.Adam(classifier.parameters(),
lr=args.learning_rate)
```

The training loop

The training loop uses the objects from the initial instantiation to update the model parameters so that it improves over time. More specifically, the training loop is composed of two loops: an inner loop over minibatches in the dataset, and an outer loop, which repeats the inner loop a number of times. In the inner loop, losses are computed for each minibatch, and the optimizer is used to update the model parameters. Example 3-21 presents the code; a more thorough walkthrough of what's going on follows.

Example 3-21. A bare-bones training loop

```
for epoch_index in range(args.num_epochs):
    train_state['epoch_index'] = epoch_index

    # Iterate over training dataset

    # setup: batch generator, set loss and acc to 0, set train mode
on
    dataset.set_split('train')
    batch_generator = generate_batches(dataset,
                                       batch_size=args.batch_size,
                                       device=args.device)
    running_loss = 0.0
    running_acc = 0.0
    classifier.train()

    for batch_index, batch_dict in enumerate(batch_generator):
        # the training routine is 5 steps:

        # step 1. zero the gradients
```

```
        optimizer.zero_grad()

        # step 2. compute the output
        y_pred = classifier(x_in=batch_dict['x_data'].float())

        # step 3. compute the loss
        loss = loss_func(y_pred, batch_dict['y_target'].float())
        loss_batch = loss.item()
        running_loss += (loss_batch - running_loss) / (batch_index +
1)

        # step 4. use loss to produce gradients
        loss.backward()

        # step 5. use optimizer to take gradient step
        optimizer.step()

        # ----------------------------------------
        # compute the accuracy
        acc_batch = compute_accuracy(y_pred, batch_dict['y_target'])
        running_acc += (acc_batch - running_acc) / (batch_index + 1)

    train_state['train_loss'].append(running_loss)
    train_state['train_acc'].append(running_acc)

    # Iterate over val dataset

    # setup: batch generator, set loss and acc to 0, set eval mode on
    dataset.set_split('val')
    batch_generator = generate_batches(dataset,
                                       batch_size=args.batch_size,
                                       device=args.device)
    running_loss = 0.
    running_acc = 0.
    classifier.eval()

    for batch_index, batch_dict in enumerate(batch_generator):

        # step 1. compute the output
        y_pred = classifier(x_in=batch_dict['x_data'].float())
```

```
        # step 2. compute the loss
        loss = loss_func(y_pred, batch_dict['y_target'].float())
        loss_batch = loss.item()
        running_loss += (loss_batch - running_loss) / (batch_index +
1)

        # step 3. compute the accuracy
        acc_batch = compute_accuracy(y_pred, batch_dict['y_target'])
        running_acc += (acc_batch - running_acc) / (batch_index + 1)

    train_state['val_loss'].append(running_loss)
    train_state['val_acc'].append(running_acc)
```

In the first line, we use a `for` loop, which ranges over the epochs. The number of epochs is a hyperparameter that you can set. It controls how many passes over the dataset the training routine should do. In practice, you should use something like an early stopping criterion to terminate this loop before it reaches the end. In the supplementary material, we show how you can do this.

At the top of the `for` loop, several routine definitions and instantiations take place. First, the training state's epoch index is set. Then, the split of the dataset is set (to `'train'` at first, then to `'val'` later when we want to measure model performance at the end of the epoch, and finally to `'test'` when we want to evaluate the model's final performance). Given how we've constructed our dataset, the split should always be set before `generate_batches()` is called. After the `batch_generator` is created, two floats are instantiated for tracking the loss and accuracy from batch to batch. For more details about the "running mean formula" we use here, we refer you to the "moving average" Wikipedia page. Finally, we call the classifier's `.train()` method to indicate that the model is in "training mode" and the model parameters are mutable. This also enables regularization mechanisms like dropout (see "Regularizing MLPs: Weight Regularization and Structural Regularization (or Dropout)").

The next portion of the training loop iterates over the training batches in `batch_generator` and performs the essential operations that update the model parameters. Inside each batch iteration, the optimizer's gradients are

first reset using the `optimizer.zero_grad()` method. Then, the outputs are computed from the model. Next, the loss function is used to compute the loss between the model outputs and the supervision target (the true class labels). Following this, the `loss.backward()` method is called on the loss object (not the loss function object), resulting in gradients being propagated to each parameter. Finally, the optimizer uses these propagated gradients to perform parameter updates using the `optimizer.step()` method. These five steps are the essential steps for gradient descent. Beyond this, there are a couple of additional operations for bookkeeping and tracking. Specifically, the loss and accuracy values (stored as regular Python variables) are computed and then used to update the running loss and running accuracy variables.

After the inner loop over the training split batches, there are a few more bookkeeping and instantiation operations. The training state is first updated with the final loss and accuracy values. Then, a new batch generator, running loss, and running accuracy are created. The loop over the validation data is almost identical to the training data, and so the same variables are reused. There is a major difference, though: the classifier's `.eval()` method is called, which performs the inverse operation to the classifier's `.train()` method. The `.eval()` method makes the model parameters immutable and disables dropout. The eval mode also disables computation of the loss and propagation of gradients back to the parameters. This is important because we do not want the model adjusting its parameters relative to validation data. Instead, we want this data to serve as a measure of how well the model is performing. If there is a large divergence between its measured performance on the training data versus the measured performance on the validation data, it is likely that the model is overfitting to the training data, and we should make adjustments to the model or to the training routine (such as setting up early stopping, which we use in the supplementary notebook for this example).

After iterating over the validation data and saving the resulting validation loss and accuracy values, the outer `for` loop is complete. Every training routine we implement in this book will follow a very similar design pattern. In fact, all gradient descent algorithms follow similar design patterns. After